BANGLADESH GOVERNMENT IT EXAMINATION

CRACK THE

IT WRITTEN

Linux • Digital Electronics • K-Map

Microprocessor & Assembly • SDLC

A complete written + MCQ preparation guide, built for the student who has to start from zero and still pass.

What is inside this book

PART 0 — 4 pages The Final Revision Sheet. Everything compressed. Read this the night before the exam.

PART 1 Basic Linux — architecture, distros, disks, permissions, and the full command list.

PART 2 Digital Electronics — number systems, codes, Boolean algebra, gates, adders, MUX, flip-flops, counters.

PART 3 The K-Map Tutorial — taught step by step, from the very first grid to don't-care and POS problems.

PART 4 Microprocessor & Assembly — 8085, 8086, registers, addressing modes, instruction set, interrupts.

PART 5 Software Engineering & SDLC — all models, requirement engineering, testing, design.

PART 6 Past Question Bank — solved, sorted by topic, with the exam name marked.

Source material: *Compact IT Written*, Basic Linux → Software Engineering chapters (pp. 525–619).

Past-paper tags shown as [45th BCS, CSE 2024] come directly from that book.

How to use this book (read this first — 2 minutes)

You said you are a weak student. Good — that is honest, and honesty is the thing that fixes it. Weak students fail these exams for one reason: they try to *read* everything and *practise* nothing. This book is built so you cannot make that mistake.

The only study method you need

1. **Week 1–2:** Read Part 1 and Part 2 slowly. Do not memorise. Just understand.
2. **Week 3:** Do Part 3 (K-map) with a pen and paper. Solve every worked example *yourself first*, then check. This is the single highest-scoring topic.
3. **Week 4:** Part 4 and Part 5. These are mostly writing/theory marks — learn the *lists* and the *difference tables*, because that is exactly how they are asked.
4. **Every day:** 20 minutes on Part 6 (question bank). Cover the answer with paper.
5. **Night before:** Part 0 only. Four pages. Nothing else.

What actually comes in the exam

From the past-paper tags in the source book, this is roughly where the marks sit:

Topic	Weight	Usually asked as
Number system conversion	**Very high**	Direct sums, 2's complement
K-map simplification	**Very high**	"Simplify using K-map + draw circuit"
Boolean simplification	**Very high**	Algebraic simplification, De Morgan
Logic gates / universal gate	High	"Implement X using only NAND"
Linux commands	High	"Write the command for . . . "
SDLC phases & models	High	7 phases, Waterfall vs Agile
Testing (black/white box)	High	Difference table
Adder / MUX / decoder	Medium	Truth table + logic diagram
Flip-flops & counters	Medium	Characteristic table, difference
8085/8086, addressing modes	Medium	Features list, mode examples

A promise about the maths

Every K-map and every conversion in this book is solved **line by line**, the way a teacher would do it on a board — no skipped steps. If you can follow arithmetic, you can do these. There is no hidden talent involved.

PART 0 — THE 4-PAGE FINAL REVISION SHEET

Read *only* these four pages the night before the exam. Everything else in this book exists to make these four pages make sense.

1 · LINUX

Facts. Linux = kernel (1991, Linus Torvalds, Univ. of Helsinki, Finland, age 21). Written in **C and assembly**. Licence **GPL**. Unix is a complete OS by AT&T; Linux is only the kernel. Layers, inside → out: **Hardware** → **Kernel** → **Shell** → **Applications** → **User**.

Features (6 marks question). Multiuser, Multitasking, Portability, Security (authentication / authorisation / encryption), GUI (X Window), Hierarchical file system, Open source.

Distros. Ubuntu (from Debian), Linux Mint (from Ubuntu), Debian (1993, most stable), Fedora / CentOS / RHEL (Red Hat family), openSUSE.

Package managers. **RPM** = Red Hat Package Manager. **YUM** = Yellowdog Updater Modified (uses RPM). **LILO** = Linux boot loader. **OpenSSH** = secure remote login suite.

Permissions. Read, Write, Execute. `chmod 777 file` ⇒ rwx for owner/group/others. r=4, w=2, x=1.

Partition tools. fdisk (CLI), parted (CLI, bigger disks), gparted (GUI).

GPT vs MBR. GPT = GUID Partition Table, part of UEFI. GPT: >2 TB disks, **128** primary partitions, GUIDs, CRC32 checksum. MBR: max 2 TB, only **4** primary partitions.

Disk layers. Physical disk (/dev/sda) → Partition → File system (ext4, xfs, btrfs) → LVM (pv, vg, lv) → RAID (mdadm) → Mounting.

Command list — learn this table, it is 5–10 marks every year

`pwd` — print working directory
`cd /` `cd ~` `cd -` `cd ..` — root / home / previous / parent
`ls` `ls -a` `ls -l` `ls -li` `ls -R` `ls -lt` `ls -lX`
`ls -a` — show hidden files (start with .)
`ls -lh` — long list, human-readable size
`mkdir dir` `rmdir dir` (must be empty)
`rm -r dir` — delete directory + its files
`touch f` — create empty file
`cat f` `cat > f` — show / create
`tac f` — show file in reverse
`cp a b` `cp -r dir /home/admin` — copy dir + contents
`mv old new` — move OR rename
`head f` — first 10 lines `head -n 7 f`
`tail f` — last 10 lines `tail -n 5 f`
`tail -f log` — last 10 lines, live updating
`sed -n '11,15p' myfile` — print line 11 to 15
`grep pattern file` — search a string
`history` — previously used commands
`cut -d ',' -f 2 f` `tr 'a-z' 'A-Z'` `wc f`
`gzip f` `zcat f.gz` — compress / view
`chmod 777 f` `chmod +rwx f` `chmod +x f`
`chown user:group f` — ownership
`useradd u` `userdel u` `passwd u`
`df -h` — disk space, human readable
`find . -name file1.txt` `echo "text"`
`ps` — your processes `top` — all processes
`ping host` `ip a` `ifconfig` `host domain`
`ssh user@ip` `scp` `wget URL`
`reboot` `poweroff` `cal` `env` `cmp f1 f2`

Shortcuts. Ctrl+C halt · Ctrl+Z suspend (fg/bg) · Ctrl+D logout · Ctrl+W erase word · Ctrl+U erase line · Ctrl+R recall command.

2 · NUMBER SYSTEMS & CODES

Bases. Decimal 10, Binary 2, Octal 8, Hex 16. 4 bits = 1 **nibble**; 8 bits = 1 **byte**.

To decimal: multiply each digit by base^{position}. Integer positions count $0, 1, 2\ldots$ from the right; fraction positions $-1, -2\ldots$ from the point.

From decimal: integer part = divide repeatedly by base, read remainders **bottom to top**. Fraction part = multiply repeatedly by base, read integers **top to bottom**.

Binary ↔ Octal: group in **3** bits from the point outwards. **Binary ↔ Hex:** group in **4** bits. Pad with zeros — left for integer part, right for fraction part.

Octal ↔ Hex: always go through binary. Never do it directly.

Hex digits. A=10, B=11, C=12, D=13, E=14, F=15.

1's complement = flip every bit.
2's complement = 1's complement + 1.
Doing it twice returns the original number.
Subtraction $A - B = A +$ (2's complement of B), then discard the final carry.

Codes. **BCD (8421)** — each *decimal digit* becomes 4 bits. If a group > 9, add 0110 (6).
Excess-3 = BCD + 0011. Un-weighted, **self-complementing**.
Gray code — only **one bit changes** between neighbours (this is why K-maps use 00,01,11,10).
ASCII — 7 or 8 bits per character. **Unicode** — 16 bits, >65,000 characters.
Parity bit — one extra bit making the total number of 1s even (even parity) or odd. Detects a single-bit error; cannot correct it.

Dec	Bin	Oct	Hex	Ex-3	Dec	Bin	Oct	Hex	Ex-3
0	0000	0	0	0011	8	1000	10	8	1011
1	0001	1	1	0100	9	1001	11	9	1100
2	0010	2	2	0101	10	1010	12	A	—
3	0011	3	3	0110	11	1011	13	B	—
4	0100	4	4	0111	12	1100	14	C	—
5	0101	5	5	1000	13	1101	15	D	—
6	0110	6	6	1001	14	1110	16	E	—
7	0111	7	7	1010	15	1111	17	F	—

3 · BOOLEAN ALGEBRA

$A + 0 = A \quad A \cdot 1 = A$
$A + 1 = 1 \quad A \cdot 0 = 0$
$A + A = A \quad A \cdot A = A$
$A + \overline{A} = 1 \quad A \cdot \overline{A} = 0$
$\overline{\overline{A}} = A$

Absorption
$A + AB = A$
$A(A + B) = A$
$A + \overline{A}B = A + B$

Commutative
$A + B = B + A$

Distributive
$A(B + C) = AB + AC$
$A + BC = (A + B)(A + C)$

De Morgan
$\overline{A + B} = \overline{A} \cdot \overline{B}$
$\overline{A \cdot B} = \overline{A} + \overline{B}$

The K-map procedure — 8 lines, memorise them

1. Draw the grid. 2 var $= 2 \times 2$, 3 var $= 2 \times 4$, 4 var $= 4 \times 4$.
2. Label headings in **Gray order: 00, 01, 11, 10**. Never 00,01,10,11.
3. SOP (Σm): put **1** in every minterm cell. POS (ΠM): put **0** in every maxterm cell.
4. Group only in powers of two: 16, 8, 4, 2, 1. Groups must be rectangles, never diagonal or L-shaped.
5. Groups **may overlap**, and **must be as large as possible**. Make as **few** groups as you can.
6. The map **wraps around** — left edge touches right edge, top touches bottom, and the four corners form one group.
7. **X (don't care)** may be used to *enlarge* a group, but never make a group only of X.
8. Read each group: keep the variable that stays constant, drop the one that changes. SOP: $1 \to A$, $0 \to \overline{A}$. POS: $0 \to A$, $1 \to \overline{A}$, and terms are ORed inside, ANDed outside.

4 · LOGIC GATES & CIRCUITS

Basic: AND, OR, NOT. **Universal:** NAND, NOR. **Special:** Ex-OR, Ex-NOR.

Why universal? Any circuit at all can be built from NAND alone (or NOR alone), so manufacturing needs only one chip type. **NOT** = both inputs of a NAND tied together. **AND** = NAND then NAND-inverter. **OR** = invert both inputs, then NAND.

Ex-OR: output 1 when inputs are **different**. $Y = A \oplus B = A\overline{B} + \overline{A}B$. **Ex-NOR:** 1 when inputs are the **same**.

Half adder. 2 inputs A, B. $S = A \oplus B$, $C = AB$. One XOR + one AND.
Full adder. 3 inputs A, B, C_{in}.
$S = A \oplus B \oplus C_{in}$
$C_{out} = AB + BC_{in} + C_{in}A$
Built from **two half adders + one OR gate**.

MUX (many → one). 2^n inputs, n select lines, 1 output. 4×1 MUX: $Y = \overline{S_1}\,\overline{S_0}I_0 + \overline{S_1}S_0I_1 + S_1\overline{S_0}I_2 + S_1S_0I_3$. **DEMUX** is the reverse: 1 input → 2^n outputs.

Encoder 2^n inputs → n outputs (built from OR gates).
Decoder n inputs → 2^n outputs (AND + NOT gates). 3-to-8 decoder: $D_0 = \overline{A}\,\overline{B}\,\overline{C}\ldots\ D_7 = ABC$.

Combinational vs Sequential.
Combinational: output depends **only** on present input, **no memory**, **no feedback**. Examples: adder, MUX, decoder.
Sequential: output depends on present input **and** past state, **has memory**, **has feedback**, usually clocked. Examples: flip-flop, counter, register.

Flip-flop	Characteristic equation	Behaviour
SR	$Q(t{+}1) = S + \overline{R}Q(t)$	$S{=}R{=}1$ is **invalid/undefined**
JK	$Q(t{+}1) = J\overline{Q} + \overline{K}Q$	Same as SR but $J{=}K{=}1$ **toggles**
D	$Q(t{+}1) = D$	Output simply follows input (**delay**/data)
T	$Q(t{+}1) = T \oplus Q(t)$	$T{=}1$ toggles, $T{=}0$ holds

Latch vs flip-flop: a latch is **level**-sensitive (enable), a flip-flop is **edge**-sensitive (clock). **Race-around** happens in a JK when $J{=}K{=}1$ and the pulse is too long — cured by a **master–slave** JK. **Counters:** *Asynchronous/ripple* — clock goes only to the first flip-flop, output of one clocks the next; slower. *Synchronous* — one clock reaches all flip-flops; faster. An n flip-flop counter counts 0 to $2^n - 1$.

5 · MICROPROCESSOR & ASSEMBLY

Definition. Controlling unit of a micro-computer on a single chip; does ALU operations and talks to connected devices. It is the **heart of the computer**.

Three buses: data, address, control. **IPC** = instructions per cycle. **Instruction set** = the group of commands the processor understands; interface between hardware and software.

8085 — "eighty-eighty-five", Intel, **1977**, NMOS. **8-bit** data bus, **16-bit** address bus ⇒ addresses **64 KB**. 16-bit PC, 16-bit SP, six 8-bit registers in pairs **BC, DE, HL**. +5 V, 3.2 MHz.

8086 — first **16-bit** processor. 16-bit ALU, registers and data bus. 20-bit address bus ⇒ 1 MB. Has a 6-byte **instruction queue** (pipelining). Versions: 8086 = 5 MHz, 8086-2 = 8 MHz, 8086-1 = 10 MHz.

8 addressing modes of 8086. Register (`MOV AX,CX`) · Immediate (`MOV AL,35H`) · Direct (`MOV AX,[1592H]`) · Register indirect (`MOV AX,[BX]`) · Based (`MOV DX,[BX+04]`) · Indexed (`MOV BX,[SI+16]`) · Based-index · Relative based-index.

Instruction groups. Data transfer (MOV, PUSH, POP, XCHG, IN, OUT) · Arithmetic (ADD, SUB, MUL, DIV, INC, DEC, CMP) · Logical (AND, OR, XOR, NOT, TEST) · Rotate (RCL, RCR, ROL, ROR) · Branch (JMP, CALL, RET) · Control.

Assembly statement: `[label:] mnemonic [operands] ; comment`. Comment starts with ;. An **assembler** converts assembly → machine code.

Interrupt = temporary halt of execution so a peripheral can be served; handled by an **ISR**. Types: hardware and software (also maskable / non-maskable).

Microprocessor	Microcontroller
Heart of a **computer system**; used for big applications	Heart of an **embedded system**; runs a single task
Only a processor — memory and I/O connected **externally**	CPU + memory + I/O all on **one chip**
High cost, high power, no power-saving mode	Low cost, low power, has power-saving mode
Used in personal computers	Used in washing machines, MP3 players

Macro vs procedure: a macro's machine code is generated **each time** it is called (more memory, no CALL/RET, for <10 instructions); a procedure's code is generated **once** (less memory, needs CALL/RET, for >10 instructions).

6 · SOFTWARE ENGINEERING & SDLC

Software engineering = analysing user requirements, then designing, building and testing software that satisfies them.

Characteristics of good software (3): **Operational**, **Transitional**, **Maintenance**.

Four key process activities: specification, development, validation, evolution.

The 7 SDLC phases — *learn this order, it is asked almost every year*:

1. Planning & requirement analysis
2. Defining requirements (produces the **SRS**)
3. Designing the software
4. Developing / coding
5. Testing
6. Deployment
7. Maintenance

User acceptance is confirmed at the **testing** stage (UAT) before deployment.

Requirement engineering process (5): feasibility study → elicitation & analysis → specification (SRS) → validation → management.

Feasibility study (3 types): **Technical** (do we have the technology?), **Operational/Behavioural** (will people actually use it?), **Economic** (will it pay?).

Models.
Waterfall — linear, each phase fixed, simple, easy to control; but high risk, cannot accept changing requirements, no early prototype. Good only for small, well-understood projects.
V-Model — waterfall bent into a V; every development stage has a matching test stage. Rigid.
Incremental — build in pieces, each piece delivered working.
Spiral — iterative with explicit **risk analysis** each loop; for large, high-risk projects.
RAD — quick prototypes, refined into production code; needs reusable components.
Prototype — build a sample first to fix the requirements.
Agile — small incremental builds, each iteration 1–3 weeks. Phases: requirements gathering, design, construction/iteration, testing/QA, deployment, feedback. Methods: **Scrum**, Crystal, DSDM, FDD, Lean, **XP**.

Agile vs Waterfall (one line): Agile allows requirement change after development starts and tests *alongside* coding; Waterfall is rigid and tests *after* the build step.

Black box	White box	Grey box
Internal code **not** known; only GUI needed	Internal code **fully** known (done by developer)	**Partial** knowledge of internals
Not used for algorithm testing	Best suited for algorithm testing	Not used for algorithm testing
Least time-consuming	**Most** time-consuming	Less than white box
Tester, developer **and end user** can take part	Only tester and developer	Tester, developer and end user

Verification vs Validation. Verification — "are we building the product **right**?", **static** testing, no code executed, comes under **QA**, done **first**. Validation — "are we building the **right** product?", **dynamic** testing, code *is* executed, comes under **QC**, done after verification.

Alpha vs Beta. Alpha — in-house, before release, uses *both* black and white box, long cycles. Beta — at the customer site, *only* black box, a few weeks, at marketing time.

Levels of testing. Unit → Integration → System → Acceptance.

Modularity. Splitting software into separately named, addressed modules that are integrated later. It is the property that makes a program intellectually manageable.
Advantages: parallel work by many people, reusable library routines, easier testing, more readable.
Disadvantages: longer execution and compile time, more storage, inter-module communication problems.

Cohesion vs Coupling — the classic one-liner: **Cohesion** measures the strength *inside* one module; **Coupling** measures the dependence *between* modules. **Good design = high cohesion, low coupling.**

SQA = the planned activities that make sure the process and product meet the required standards. **Six Sigma** methodologies: **DMAIC** (improve existing) and **DMADV** (design new).

Last five minutes — the things students most often lose marks on

- K-map headings must be **00, 01, 11, 10**. Writing 00,01,10,11 destroys every answer on the page.
- In 2's complement subtraction, **discard the final carry** — do not write it in the answer.
- For BCD addition, add **0110** only to a group that exceeds 9, then propagate the carry.
- "Simplify using K-map **and draw the logic circuit**" — the circuit carries half the marks. Always draw it.
- SDLC has **7** phases and **SRS** is produced in phase **2**, not phase 1.
- Write difference questions as a **two-column table**, not a paragraph. Examiners award per row.

PART 1

BASIC LINUX

What they ask from this chapter

- Write the Linux command for . . . [BTCL AME 2023, MGMCL AM(ICT) 2022, NESCO AM(ICT) 2021, BPSC SAE 2021, APSCL AE(ICT) 2021, BGDCL AE(CSE) 2021, BCC AP 2021, Titas Gas AE(CSE) 2024, Milk Vita AM(CSE) 2023]
- Difference between Linux and Unix / Linux and Windows
- What is kernel, shell, LILO, CLI, RPM, YUM, OpenSSH?
- Linux features and benefits (a standard 5-mark written question)

1.1 What Linux actually is

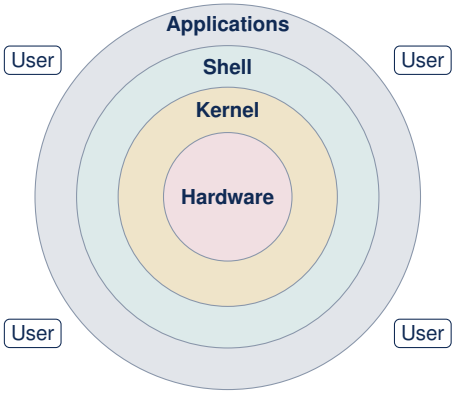
Linux is an **operating system** — or more precisely a **kernel** — distributed under an open-source licence. Its functionality list is very much like UNIX.

The **kernel** is the program at the heart of the system. It takes care of the fundamental work: letting the hardware communicate with the software, managing memory, processes and devices.

History — the one-line answer

In **April 1991**, **Linus Torvalds**, a 21-year-old computer science student at the **University of Helsinki, Finland**, began a personal project to create a new operating system kernel. That project became Linux.

Linux system architecture



Read it from the inside out: the **hardware** is wrapped by the **kernel**, the kernel is wrapped by the **shell** (the command interpreter), then the **applications**, and the **users** sit outside everything. A user never touches the hardware directly — every request passes inward through these layers.

Benefits and features

Benefits of using Linux

- Open source — anyone with programming knowledge can modify it.
- Easy to learn for beginners.
- Millions of programs and applications available, most of them free.
- The OS of choice for **server** environments, because of stability and reliability.
- Lightweight — needs far fewer resources than other operating systems.
- Very flexible — desktop, embedded systems and servers.
- Powerful networking support; client–server systems are easy to set up.
- Multitasking — runs many tasks at once without slowing down.

Linux features (memorise the 6 headings)

- **Multiuser capability** — many users share the same resources (memory, disk) from different terminals.
- **Multitasking** — more than one function at a time, by dividing CPU time intelligently.
- **Portability** — *not* about file size; it means it supports many different types of hardware.
- **Security** — three ways: *authentication* (password + login ID), *authorisation* (read/write/execute permission), *encryption* (converts the file to unreadable form).
- **GUI (X Window system)** — Linux is command-line based but can be made graphical by installing packages.
- **File system** — hierarchical; files and directories are arranged in a tree.

Vocabulary they ask for definitions

Kernel	The core program of the OS; manages hardware, memory and processes.
Shell	The command interpreter between the user and the kernel (e.g. bash).
Utilities	Utility programs that give the user most of the OS functionality.
LILO	*Linux Loader* — a boot loader that loads Linux into main memory to begin operation.
CLI	*Command Line Interface* — lets the user type declarative commands to instruct the computer.
RPM	*Red Hat Package Manager* — builds, installs, upgrades, queries and removes packages on Red Hat based systems (RHEL, Fedora, CentOS, Rocky, openSUSE).
YUM	*Yellowdog Updater, Modified* — an open-source package management utility that uses the RPM format (CentOS, RHEL, Fedora).
OpenSSH	*Open Secure Shell* — a suite of tools giving secure communication over an unsecured network by encrypting traffic; used for remote login, file transfer and tunnelling.
Shell variable	A character string holding a value (number, filename, text). The shell can create, delete and assign variables.

Distributions

A **Linux distribution** is an OS built from the Linux kernel plus a package manager, GNU libraries and tools, a window system, a desktop environment and documentation.

- **Ubuntu** — the most well known; built *from Debian*; easy graphical desktop, many pre-installed apps.
- **Linux Mint** — based on *Ubuntu*, shares its repositories.
- **Debian** — exists since **1993**, releases slowly, therefore the *most stable*. Ubuntu is based on it.
- **Fedora / CentOS / RHEL** — the Red Hat family. RHEL is the commercial one.

1.2 Disks, partitions and file systems

Partitioning tools

- **fdisk** — command-line tool for creating and managing partitions.
- **parted** — also command-line, more versatile, supports larger disks.
- **gparted** — the graphical version.

Disk management layers (answer in this order)

- **Physical disk (block device)** — the raw disk, e.g. `/dev/sda`.
- **Partition level** — a disk split into partitions, each with its own file system and mount point.
- **File system level** — partitions formatted as ext4, xfs, btrfs. Tools: mkfs, mount, umount.
- **LVM** — Logical Volume Management; an abstraction layer over disks. Components: *pv* (physical volume), *vg* (volume group), *lv* (logical volume).
- **RAID** — Redundant Array of Independent Disks; combines several disks into one logical unit for fault tolerance and performance. Managed with `mdadm`.
- **Mounting** — attaching a partition to a directory, e.g. `mount /dev/sda1 /mnt/mydisk`.

GPT vs MBR

GPT = GUID Partition Table. It is part of the **UEFI** specification and replaces the older **MBR** (Master Boot Record).

	GPT	MBR
Disk size	more than 2 TB	limited to 2 TB
Primary partitions	up to **128**	only **4**
Identifiers	globally unique GUIDs	none
Integrity	**CRC32** checksum	none

File permissions — 3 types

Read — can read the file and list the directory.
Write — can write new files in the directory.
Execute — can access and run the file in a directory.

Open-source web servers: Apache HTTP Server, Nginx, Caddy.

1.3 Linux vs Unix vs Windows

Linux	Unix
Open source, freely available to everyone	Can only be used by its copyright holders
In great demand — any home user, developer or student can use it	Developed mainly for servers, workstations and mainframes
Developed by worldwide developers sharing code	Developed by AT&T Labs, commercial vendors and non-profits
Supports more file systems than Unix	Supports file systems, but fewer
A Unix *clone* — behaves like Unix but contains none of its code	Contains completely different code developed by AT&T
Linux is just the **kernel**	Unix is a **complete package** of an operating system

Linux	Windows
Users can access and alter the kernel source code	Usually users cannot access the source code
Many highly customisable distributions	Fewer options to customise
Written in **assembly language and C**	Written in **C++ and assembly**
Huge community support through forums and online search	Free and paid support, easily accessible online forum
Distributed under the **GPL** (GNU General Public License)	Distributed under a **proprietary** commercial licence

1.4 The command reference

This section is where the marks are. Do not just read it — write each command out by hand once.

Navigation and listing

`pwd`	Print working directory — shows the full path from root to where you are.
`cd <dir>`	Change directory.
`cd ~`	Go to your home directory.
`cd --`	Go back to the previous directory.
`cd ..`	Go up to the parent directory.
`cd /`	Go to the system root directory.
`cd ~username`	Go to that user's home directory.
`cd 'Dir name with space'`	Quote the path, or escape with backslash: `cd Dir\ name`.
`cd Dir1/Dir2/Dir3`	Descend several sub-directories at once.
`ls`	List the contents of the current directory.
`ls -a`	**Include hidden files** (in Linux, hidden files begin with a dot).
`ls -l`	Long list format (permissions, owner, size, date).
`ls -li`	Prints the index (inode) number in the first column.
`ls -p`	Marks directories with a trailing slash.
`ls -r`	Print the list in reverse order.
`ls -R`	Also display the contents of the sub-directories.
`ls -lX`	Group files with the same extension together.
`ls -lt`	Sort by time, most recently modified at the top.
`ls -lh`	Long list with **human-readable** size — this is the "human readable size" answer.

Creating, copying, deleting

`mkdir <dirname>`	Make a directory. ("Make a directory named PSC" → `mkdir PSC`)
`rmdir <dirname>`	Delete a directory — **only if it is empty**.
`rm -r <dirname>`	Delete a directory **and all the files inside it**.
`rm <file>`	Remove a file.
`touch <file>`	Create an empty file. `touch f1 f2 f3` creates several at once.
`cat <file>`	Display the contents of a file.
`cat > <file>`	Create a file and type into it; press **Ctrl+D** to save.
`tac <file>`	The reverse of cat — displays the file from the last line upwards.
`cp <old> <new>`	Copy a file within the same directory.
`cp -r PSC /home/admin`	Copy a directory **with all its contents** into another directory.
`mv <old> <new>`	Move a file **or rename** it — one command does both.
`rename 's/old/new/' files`	Rename a large group of files at once.

Reading, filtering, searching

`head <file>`	First **10** lines.
`head -n 7 apscl.txt`	First **7** lines — the exact APSCL 2021 answer.
`tail <file>`	Last **10** lines.
`tail -n 5 file.txt`	**Last five lines of a text file**.
`tail -f logfile`	Last 10 lines of a log that is **continuously updating** (Titas Gas 2024 answer).
`sed -n '11,15p' myfile`	Show **line 11 to line 15** of a file.
`grep [options] pattern file`	Search a file for a pattern and print matching lines. *grep* = global search for regular expression and print out.
`history`	Show all the recently used commands. Combine: `history | grep ssh`.
`cut -d ',' -f 2 file`	Select a column. `-d` sets the delimiter, `-f` the column number.
`tr 'a-z' 'A-Z'`	Translate file content, e.g. lower case to upper case.
`wc <file>`	Count lines, words and characters.
`find . -name file1.txt`	Search for files in a directory hierarchy.
`cmp file1 file2`	Byte-by-byte comparison of two files.
`cksum README.txt`	Show the CRC checksum and byte count of a file.

Permissions, users, system

`chmod 777 myfile.txt`	Give **read, write and execute** to owner, group and others.
`chmod +rwx <file>`	Add all permissions. `chmod -rwx` removes them.
`chmod +x <file>`	Make a file executable. `chmod -wx` removes write and execute.
`chown user:group <file>`	Change ownership and group — the "ownership and group permission" answer.
`useradd <user>`	Create a new user account (the "user creation" answer).
`userdel <user>`	Delete a user account.
`passwd <user>`	Create or change a user's password.
`df` `df -h`	Show disk space usage; `-h` makes it **human readable**.
`ps` `top`	Your active processes / all running processes.
`env`	List all current environment variables.
`echo "text"`	Display a line of text.
`cal`	Print a calendar.
`reboot` `poweroff`	Restart / shut down the system.

Networking, transfer, compression

`ip a` or `ip addr`	The modern replacement for ifconfig — assign an IP, enable or disable an interface.
`ifconfig`	Configure, view and control network interfaces. `sudo ifconfig eth0 up` / `down` / `sudo ifconfig eth0 172.16.25.125`.
`ping <destination>`	Check connectivity between two nodes. *Packet Internet Groper*.
`ssh user@host`	Create a remote connection through the SSH protocol.
`scp`	Secure copy — remote file copy program.
`wget http://site/file.txt`	**Download a file from a URL**.
`host <domain>`	Show the IP for a domain name and vice versa (a DNS lookup).
`gzip f1 f2`	Compress files; replaces the original with a `.gz` file.
`zcat <file.gz>`	Display a compressed file without unpacking it.

Keyboard shortcuts (a free MCQ mark)

Ctrl+C halt the current command · **Ctrl+Z** stop it, resume with `fg` or `bg` · **Ctrl+D** log out of the session (same as `exit`) · **Ctrl+W** erase one word · **Ctrl+U** erase the whole line · **Ctrl+R** search and recall a recent command.

Model answer — a real past question, fully solved

[APSCL AE(ICT) 2021] *Write the Linux command:* (a) create a file `apscl.txt` in the current location, (b) give read, write and execute permission to all, (c) read the first 7 lines, (d) delete the file.

(a) `touch apscl.txt` or `cat > apscl.txt`

(b) `chmod 777 apscl.txt`

(c) `head -n 7 apscl.txt`

(d) `rm apscl.txt`

[BGDCL AE(CSE) 2021] *List home directory files with all details and human-readable size, then display only the 10th to 15th lines of the result, in one execution:*

```
ls -lh ~ | sed -n '10,15p'
```

PART 2

DIGITAL ELECTRONICS I — NUMBERS, CODES, BOOLEAN

What they ask from this chapter

- Convert between decimal, binary, octal and hexadecimal [45th BCS CSE/ICT 2024, BPDB AE(CSE) 2024, BTCL JAM 2024, 18th NTRCA 2024, BPSC AME 2021, DPDC 2020, ICT Ministry AP 2017]
- Write the 2's complement of $(+55)_{10}$ [45th BCS CSE 2024]
- BCD addition, Excess-3 code, parity for error detection [NPCNL JAM 2022, DPDC 2020, BB AP 2023]
- Simplify a Boolean function / state De Morgan's law [45th BCS CSE 2024, BPSC AP 2023, BPSC ADA 2022, BPSC SAE 2021]

2.1 The number systems

A **number system** is a technique for representing and working with numbers. Number systems are either **non-positional** or **positional**. All four systems you need are positional, which simply means the *place* of a digit decides its value.

System	Base	Digits used
Decimal	10	0–9
Binary	2	0, 1 (a binary digit is a **bit**)
Octal	8	0–7
Hexadecimal	16	0–9 then A=10, B=11, C=12, D=13, E=14, F=15

Words they define in MCQs

Bit — the smallest unit of data, a single 0 or 1. **Nibble** — 4 bits (half a byte). **Byte** — 8 bits (also called an *octet*). **Base** — how many different digits the system has.

Rule 1 — any base → decimal

Multiply each digit by base^{position}, then add. In the integer part positions count $0, 1, 2\ldots$ from the right. In the fractional part they count $-1, -2\ldots$ from the point rightwards.

$$(1101.11)_2 = (1\times 2^3)+(1\times 2^2)+(0\times 2^1)+(1\times 2^0)+(1\times 2^{-1})+(1\times 2^{-2})$$
$$= 8+4+0+1+0.5+0.25 = (\mathbf{13.75})_{\mathbf{10}}$$

$$(145.23)_8 = (1\times 8^2)+(4\times 8^1)+(5\times 8^0)+(2\times 8^{-1})+(3\times 8^{-2})$$
$$= 64+32+5+0.25+0.05 = (\mathbf{101.3})_{\mathbf{10}}$$

$$(1A5.2)_{16} = (1\times 16^2)+(10\times 16^1)+(5\times 16^0)+(2\times 16^{-1}) = 256+160+5+0.125 = (\mathbf{421.125})_{\mathbf{10}}$$

Rule 2 — decimal → any base

The two halves work in opposite directions — this is the step students get wrong

Integer part: divide repeatedly by the base. Collect the remainders and read them **from bottom to top** (last remainder is the MSB).
Fraction part: multiply repeatedly by the base. Collect the integer parts and read them **from top to bottom** (first one is the MSB).

Worked example: $(423.150)_{10} = (?)_8$ [BPDB AE(CSE) 2024]

Integer part
$423 \div 8 = 52$ remainder **7** (LSB)
$52 \div 8 = 6$ remainder **4**
$6 \div 8 = 0$ remainder **6** (MSB)
Read bottom to top: $(423)_{10} = (647)_8$

Fraction part
$0.150 \times 8 = 1.200 \rightarrow$ **1** (MSB)
$0.200 \times 8 = 1.600 \rightarrow$ **1**
$0.600 \times 8 = 4.800 \rightarrow$ **4**
$0.800 \times 8 = 6.400 \rightarrow$ **6**
Read top to bottom: $(0.150)_{10} = (0.1146\ldots)_8$

$\therefore (423.150)_{10} = (\mathbf{647.1146\ldots})_{\mathbf{8}}$

Worked example: $(423.125)_{10} = (?)_{16}$
$423 \div 16 = 26$ r **7**; $26 \div 16 = 1$ r **10 (A)**; $1 \div 16 = 0$ r **1** $\Rightarrow (423)_{10} = (1A7)_{16}$
$0.125 \times 16 = 2.00 \rightarrow$ **2** $\Rightarrow (0.125)_{10} = (0.2)_{16}$
$\therefore (423.125)_{10} = (\mathbf{1A7.2})_{\mathbf{16}}$

Rule 3 — binary ↔ octal ↔ hexadecimal

The grouping rule

Octal ↔ binary: group in **3** bits. Hexadecimal ↔ binary: group in **4** bits.
Always group **outwards from the decimal point**. If the last group is short, pad with zeros — on the **left** for the integer part, on the **right** for the fractional part.
Octal ↔ hexadecimal has **no direct route**. Always go through binary.

Example: $(11110100010.01011101)_2 = (?)_{16}$

0111	1010	0010	.	0101	1101
↓	↓	↓		↓	↓
7	10 (A)	2	.	5	13 (D)

$\therefore (11110100010.01011101)_2 = (\mathbf{7A2.5D})_{\mathbf{16}}$

Example: $(675.24)_8 = (?)_{16}$ — go through binary.
Step 1, octal to binary (3 bits each): $6\,7\,5\,.\,2\,4 \rightarrow 110\,111\,101\,.\,010\,100$
Step 2, regroup that binary in 4 bits: $0001\,1011\,1101\,.\,0101\,0000$
Step 3, read as hex: $1\,B\,D\,.\,5\,0$ $\therefore (675.24)_8 = (\mathbf{1BD.50})_{\mathbf{16}}$

2.2 Binary arithmetic and complements

Addition rules

$0 + 0 = 0$
$0 + 1 = 1$
$1 + 0 = 1$
$1 + 1 = 0$ carry **1**
$1 + 1 + 1 = 1$ carry **1**

Subtraction rules

$0 - 0 = 0$
$1 - 0 = 1$
$0 - 1 = 1$ borrow **1**
$1 - 1 = 0$

Example: $(1011.0110)_2 + (1101.0111)_2 = (\mathbf{11000.1101})_\mathbf{2}$ (check in decimal: $11.375 + 13.4375 = 24.8125$).

Example: $(110101.101)_2 - (010110.110)_2 = (\mathbf{011111.001})_\mathbf{2}$.

1's and 2's complement

The definitions, exactly as the examiner wants them

1's complement — obtained by complementing (flipping) *all* the bits of a signed binary number. The 1's complement of a positive number gives a negative number, and vice versa.
2's complement — obtained by **adding 1 to the 1's complement**.

$$\boxed{\text{2's complement} = \text{1's complement} + 1}$$

Taking either complement **twice** returns the original signed number.

Worked example: 2's complement of -108.
Signed binary of $108 = 01101100$
1's complement $= 10010011$
2's complement $= 10010011 + 1 = \mathbf{10010100}$

Worked example: 2's complement of $(+55)_{10}$ [45th BCS CSE 2024]
55 in 8-bit signed binary $= 00110111$
1's complement $= 11001000$
2's complement $= 11001000 + 1 = \mathbf{11001001}$

Subtraction using 2's complement — the exam favourite

To compute $A - B$: take the **2's complement of B**, then **add** it to A. If a carry comes out of the last column, **discard it** — the answer is positive. If there is *no* final carry, the answer is negative, and its magnitude is the 2's complement of what you got.

Example: $(+11)_{10} - (+13)_{10}$. $11 = 00001011$, $13 = 00001101$.
2's complement of 13 $= 11110010 + 1 = 11110011$.
$00001011 + 11110011 = 11111110$ — no final carry, so it is negative.
2's complement of $11111110 = 00000001 + 1 = 00000010 = 2$. Answer: $\mathbf{-2}$.

2.3 Binary codes

BCD (8421)	*Binary Coded Decimal.* Each **decimal digit** is written as its own 4-bit group. It is a **weighted** code. Fast and efficient for converting decimal to binary.
Excess-3	BCD **+ 0011**. It has no weights, so it is an **un-weighted** code, and it is **self-complementing**.
Gray code	Only **one bit changes** between two consecutive values. This is why K-map headings run 00, 01, 11, 10 — adjacent cells differ in one variable only.
ASCII	*American Standard Code for Information Interchange*, invented by ANSI. Uses **7 or 8 bits** per character.
Unicode	Uses **16 bits**, so it represents more than **65,000** characters — far more than ASCII.
Parity bit	An extra check bit added to a block so that the total number of 1s becomes even (even parity) or odd (odd parity). It is the simplest error-*detecting* code.

Decimal	8421 (BCD)	2421	84-2-1	Excess-3
0	0000	0000	0000	0011
1	0001	0001	0111	0100
2	0010	0010	0110	0101
3	0011	0011	0101	0110
4	0100	0100	0100	0111
5	0101	1011	1011	1000
6	0110	1100	1010	1001
7	0111	1101	1001	1010
8	1000	1110	1000	1011
9	1001	1111	1111	1100

BCD addition

The BCD addition rule

Add the two BCD numbers 4 bits at a time. If any 4-bit group produces a result **greater than 9 (1001)**, or produces a carry out, then **add 0110 (decimal 6)** to that group and carry into the next group. If every group is 9 or less, the answer is already valid — add nothing.

Worked example: $00010011 + 00100110$ [NPCNL, JAM 2022]

$$\begin{array}{rll} & 0001\ 0011 & = (13)_{10} \\ + & 0010\ 0110 & = (26)_{10} \\ \hline & 0011\ 1001 & = (39)_{10} \end{array}$$

Check each group: $0011 = 3$ and $1001 = 9$. Both are ≤ 9, so **no correction is needed**. The answer **0011 1001** is a valid BCD number, and $13 + 26 = 39$ confirms it.

Excess-3 example: what is the Excess-3 code of 1010? [DPDC 2020]
1010 is not a valid BCD digit on its own; treated as decimal 10, the digits are 1 and 0, whose Excess-3 codes are 0100 and 0011, giving **0100 0011**. For a single decimal digit, just add 0011: e.g. decimal $786 \rightarrow 1010\ 1011\ 1001$.

Parity for error detection

The parity method adds one extra bit to the end of a data block. The bit is set to 0 or 1 so that the total count of 1s is even (for even parity). The receiver counts the 1s in what arrived; an odd count means an error occurred.

Example: send the 4-bit block **1011**. It contains three 1s, so the parity bit is set to **1**, and the transmitted block is **10111** (now four 1s — even). If **10110** arrives, it has two 1s — even — so it is assumed error-free. If **10101** arrives, it has three 1s — odd — so an error is detected.

ASCII bit string of "Delta" [BB AP 2023]: the ASCII codes are 68 65 6C 74 61 (hex) = 01000100 01100101 01101100 01110100 01100001.

2.4 Boolean algebra

Boolean algebra deals with binary variables and binary values. It was developed by **George Boole** in **1854**. It is also called binary algebra or logical algebra.

The laws — learn all of them, they are worth easy marks

Logical OR	Logical AND	Associative	Distributive
$A+0=A$	$A\cdot 1=A$	$A+(B+C)=(A+B)+C$	$A(B+C)=AB+AC$
$A+1=1$	$A\cdot 0=0$	$A\cdot(B\cdot C)=(A\cdot B)\cdot C$	$A+BC=(A+B)(A+C)$
$A+A=A$	$A\cdot A=A$		
$A+\overline{A}=1$	$A\cdot\overline{A}=0$	**De Morgan**	**Commutative**
$\overline{\overline{A}}=A$	$A+AB=A$	$\overline{A+B}=\overline{A}\cdot\overline{B}$	$A+B=B+A$
	$A+\overline{A}B=A+B$	$\overline{A\cdot B}=\overline{A}+\overline{B}$	$A\cdot B=B\cdot A$

De Morgan in plain language

"Break the bar and change the sign." NOT(A OR B) is the same as (NOT A) AND (NOT B); and NOT(A AND B) is the same as (NOT A) OR (NOT B).
Everyday example: "It is not the case that it is raining or windy" means exactly "it is not raining *and* it is not windy".

Proofs they ask for

Prove $A+1=1$
If $A=0$: LHS $=0+1=1=$ RHS
If $A=1$: LHS $=1+1=1=$ RHS (proved)

Prove $A+\overline{A}=1$
If $A=0$: LHS $=0+1=1=$ RHS
If $A=1$: LHS $=1+0=1=$ RHS (proved)

Worked simplifications from past papers

Example 1 — $F=A'B'C+A'BC+AB'$

[45th BCS, CSE 2024]
$F=\overline{A}\,\overline{B}C+\overline{A}BC+A\overline{B}$
Take $\overline{A}C$ common from the first two terms:
$=\overline{A}C(\overline{B}+B)+A\overline{B}$
$=\overline{A}C(1)+A\overline{B}$ (since $B+\overline{B}=1$)

$= \boxed{\overline{A}C + A\overline{B}}$

Example 2 — $Y = A\overline{B} + (A+B)C$

[BPSC, ADA 2022]

$Y = A\overline{B} + (A+B)C = A\overline{B} + AC + BC$

This is already minimal in three terms; if a consensus removal is asked, note that BC is the consensus of $A\overline{B}$ and AC only when the literals match — here they do not, so $\boxed{Y = A\overline{B} + AC + BC}$.

Example 3 — $(x + y + xy)(x + z)$

[BPSC AME 2021]

First simplify inside: $x + y + xy = x + y(1 + x) = x + y$ (since $1 + x = 1$)

So the expression is $(x + y)(x + z)$.

Using $A + BC = (A + B)(A + C)$ read backwards with $A = x$, $B = y$, $C = z$:

$(x + y)(x + z) = \boxed{x + yz}$

Example 4 — $\overline{x}\overline{y}z + \overline{x}yz + x\overline{y}$

[BPSC AP 2021]

$= \overline{x}z(\overline{y} + y) + x\overline{y} = \overline{x}z(1) + x\overline{y} = \boxed{\overline{x}z + x\overline{y}}$

Example 5 — prove $A + \overline{A}B = A + B$

$A + \overline{A}B = (A + \overline{A})(A + B)$ [distributive $A + BC = (A + B)(A + C)$]

$= 1 \cdot (A + B) = \boxed{A + B}$

This identity appears constantly. Memorise it in both directions.

Minterms, maxterms, SOP and POS

SOP (Sum of Products) — terms are ANDed inside and ORed outside, e.g. $F = AB + \overline{A}C$. It is written with Σm and built from **minterms** (rows where the output is **1**). In a minterm a variable appears plain if it is 1 and barred if it is 0.

POS (Product of Sums) — terms are ORed inside and ANDed outside, e.g. $F = (A + B)(\overline{A} + C)$. It is written with ΠM and built from **maxterms** (rows where the output is **0**). In a maxterm a variable appears plain if it is **0** and barred if it is 1 — the opposite of a minterm.

Converting SOP → POS. Change the symbol Σ to Π, then write the index numbers that were *missing* from the SOP list.

Example: $F = \Sigma m(0, 2, 3, 5, 7)$ over three variables. The missing indexes are 1, 4 and 6, so $F = \Pi M(1, 4, 6)$. Writing them out: $1 = (x + y + \overline{z})$, $4 = (\overline{x} + y + z)$, $6 = (\overline{x} + \overline{y} + z)$, giving $F = (x + y + \overline{z})(\overline{x} + y + z)(\overline{x} + \overline{y} + z)$.

Non-standard to standard SOP. Multiply each short term by the missing variable in the form $(X + \overline{X})$.

Example: $F = AB + AC + BC$

$= AB(C + \overline{C}) + A(B + \overline{B})C + (A + \overline{A})BC$

$= ABC + AB\overline{C} + ABC + A\overline{B}C + ABC + \overline{A}BC$

$= \boxed{ABC + AB\overline{C} + A\overline{B}C + \overline{A}BC}$ (duplicates removed, since $X + X = X$)

PART 3

THE K-MAP TUTORIAL

Why this chapter matters more than any other

K-map questions appear in almost every paper, they carry high marks, and — unlike theory — they are **fully mechanical**. There is nothing to understand deeply and nothing to remember creatively. If you follow eight fixed steps, you get full marks every single time. This is the chapter that turns a weak student into a passing one.

[38th BCS] [Bank SO(AME) 2021] [BPSC ADA 2022] [JBL SO(ANE) 2020] [BOF AE(CSE) 2021] [BUET M.Sc. May 2016]

3.1 Lesson 1 — what a K-map is and why it works

The **Karnaugh map** is a method of simplifying Boolean expressions. It is a table, like a truth table, but arranged so that **physically adjacent cells differ in exactly one variable**. That single fact is the whole trick: when two neighbouring cells are both 1, the variable that changed between them can be thrown away.

Why the headings are 00, 01, 11, 10 and never 00, 01, 10, 11

Look at the change from one heading to the next: $00 \to 01$ (one bit changed), $01 \to 11$ (one bit changed), $11 \to 10$ (one bit changed), and wrapping round, $10 \to 00$ (one bit changed). This is **Gray code** order.
If you wrote 00, 01, 10, 11 instead, then $01 \to 10$ changes *two* bits, adjacency breaks, and every grouping on your page becomes wrong. **This one mistake fails more students than any other.**

The three grids

2 variables

A \ B	0	1
0	m_0	m_1
1	m_2	m_3

3 variables

A \ BC	00	01	11	10
0	m_0	m_1	m_3	m_2
1	m_4	m_5	m_7	m_6

4 variables

AB \ CD	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

Notice the minterm numbers are **not** in reading order — they follow the Gray headings. For a 4-variable map, memorise the layout as four rows: **0 1 3 2** / **4 5 7 6** / **12 13 15 14** / **8 9 11 10**. Write this layout down first, every time, before you do anything else.

3.2 Lesson 2 — the eight rules

The procedure. Do not deviate from it.

Step 1. Draw the correct grid: 2 variables $= 2 \times 2$, 3 variables $= 2 \times 4$, 4 variables $= 4 \times 4$.
Step 2. Write the headings in Gray order: **00, 01, 11, 10**.
Step 3. Fill the map. For **SOP** (Σm, minterms), write **1** in each listed cell. For **POS** (ΠM, maxterms), write **0** in each listed cell. Write **X** in every don't-care cell.
Step 4. Group only in **powers of two**: 16, 8, 4, 2, or 1. A group must be a **rectangle or square** — never diagonal, never L-shaped.
Step 5. Make each group **as large as possible**, and make **as few groups as possible**. Groups **may overlap**. Every 1 must end up inside at least one group.
Step 6. The map **wraps around**. The left column is adjacent to the right column; the top row is adjacent to the bottom row; and **the four corners together form one valid group of four**.
Step 7. An **X** may be used to make a group *bigger*, or ignored entirely. Never form a group made *only* of X cells.
Step 8. Read each group. Keep the variables that **stay the same** across the whole group; drop the ones that change.

For **SOP**: a variable that stays 1 is written plain (A); one that stays 0 is written barred ($\overline{A}$). Multiply within a group, add the groups.

For **POS**: the opposite — a variable that stays **0** is written plain, one that stays **1** is barred. Add within a group, multiply the groups.

How big a group, how many literals?

In a 4-variable map: a group of **1** cell leaves 4 literals; **2** cells leaves 3 literals; **4** cells leaves 2 literals; **8** cells leaves 1 literal; **16** cells means $F = 1$.
Every time you double the group size, you delete one more variable. That is exactly why bigger groups are better — the answer gets shorter.

3.3 Lesson 3 — your first SOP map, done slowly

Problem: Simplify $f(W, X, Y, Z) = W\overline{XY} + WY + \overline{W}Y\overline{Z}$ using a K-map.

Step A — expand every term to full minterms

Each term is short, so multiply by the missing variables in the $(V + \overline{V})$ form:
$W\overline{XY}(Z + \overline{Z}) + WY(X + \overline{X}) + \overline{W}Y\overline{Z}(X + \overline{X})$
$= W\overline{XY}Z + W\overline{XY}\overline{Z} + WXY + W\overline{X}Y + \overline{W}XY\overline{Z} + \overline{W}\,\overline{X}Y\overline{Z}$
Expanding WXY and $W\overline{X}Y$ over Z as well gives the full minterm list: $m_2, m_6, m_8, m_9, m_{11}, m_{10}, m_{14}, m_{15}$.

Step B — fill the grid

WX \ YZ	00	01	11	10
00	0	0	0	1
01	0	0	0	1
11	0	0	1	1
10	1	1	1	1

Remember the row order is $WX = 00, 01, 11, 10$ and the column order $YZ = 00, 01, 11, 10$. Place a 1 in each cell whose minterm number appears in the list. Leave everything else 0.

Step C — group and read

Group 1 (four cells) — the whole bottom row, $WX = 10$, across all YZ. W stays 1, X stays 0, Y and Z both change $\Rightarrow$ term $= W\overline{X}$.

Group 2 (four cells) — the right-hand two columns of the bottom two rows ($WX = 11$ and 10, $YZ = 11$ and 10). Here $W = 1$ throughout and $Y = 1$ throughout; X and Z change $\Rightarrow$ term $= WY$.

Group 3 (four cells) — the $YZ = 10$ column, all four rows. Y stays 1, Z stays 0 $\Rightarrow$ term $= Y\overline{Z}$.

$$\therefore \boxed{f = W\overline{X} + WY + Y\overline{Z}}$$

Three terms instead of three long ones — and notice the answer is *shorter than the question*. That is what a correct K-map always does.

3.4 Lesson 4 — a POS map (zeros instead of ones)

Problem: Simplify $f(X, Y, Z) = \Pi M(0, 1, 2, 4)$ using a K-map.

The symbol ΠM tells you this is **product of maxterms**, so you write **0** in cells 0, 1, 2 and 4 and leave the rest blank.

X \ YZ	00	01	11	10
0	0	0		0
1	0			

There is no way to group eight zeros or four zeros here. There are exactly **three** groupings of **two adjacent zeros**, and after those three groups no single zero is left ungrouped.

Group 1 — cells m_0 and m_1 (top row, $YZ = 00$ and 01): $X = 0$, $Y = 0$, Z changes. For POS, 0 becomes plain: $\Rightarrow (X + Y)$.

Group 2 — cells m_0 and m_2 (top row, $YZ = 00$ and 10): $X = 0$, $Z = 0$, Y changes $\Rightarrow (X + Z)$. Written in the standard order this is $(Z + X)$.

Group 3 — cells m_0 and m_4 (column $YZ = 00$, both rows): $Y = 0$, $Z = 0$, X changes $\Rightarrow (Y + Z)$.

$$\therefore \boxed{f = (X + Y)(Y + Z)(Z + X)}$$

The single thing that goes wrong in POS problems

In SOP you write a **barred** variable when the value is 0. In POS you write a **plain** variable when the value is 0. It is reversed. Write "POS $\Rightarrow$ reversed" at the top of your answer sheet before you start.

3.5 Lesson 5 — don't-care conditions

The **don't care** condition says some input combinations can never occur, so you are free to treat those cells as either 0 or 1 — whichever makes your group bigger. They are marked **X**.

Where do they come from? Invalid combinations. In BCD, the codes 1010 to 1111 are don't cares because no decimal digit uses them. In Excess-3, the don't cares are 0000, 0001, 0010, 1101, 1110 and 1111.

Worked example 1

Problem: Minimise $F = \Sigma m(1, 5, 6, 12, 13, 14) + d(4)$ in SOP minimal form.

AB \ CD	00	01	11	10
00	0	1	0	0
01	X	1	0	1
11	1	1	0	1
10	0	0	0	0

Cells 1, 5, 13 hold 1; cells 6, 14, 12 hold 1; cell 4 holds X.

Group 1 — m_1, m_5 plus the X at m_4 lets us take m_4, m_5, m_{12}, m_{13} as a block of four.

Group 2 — m_6 and m_{14} (a pair).

Group 3 — m_{12} and m_{13} combine upward with m_4, m_5.

Reading the maximal groups:

- m_4, m_5, m_{12}, m_{13}: $B = 1$, $C = 0$ throughout, A and D change $\Rightarrow B\overline{C}$
- m_{12}, m_{13}, m_{14} region grouped as m_{12}, m_{13} with m_4, m_5 already covered; the pair m_6, m_{14} gives $B = 1$, $C = 1$, $D = 0 \Rightarrow BC\overline{D}$, which merges with m_{12}, m_{13} as $B\overline{D}$
- m_1, m_5: $A = 0$, $C = 0$, $D = 1 \Rightarrow \overline{A}\,\overline{C}D$

$\therefore \boxed{F = B\overline{C} + B\overline{D} + \overline{A}\,\overline{C}D}$

Worked example 2

Problem: Minimise $F(A, B, C, D) = \Sigma m(1, 2, 6, 7, 8, 13, 14, 15) + d(3, 5, 12)$ in SOP minimal form.

AB \ CD	00	01	11	10
00	0	1	X	1
01	0	X	1	1
11	X	1	1	1
10	1	0	0	0

Use every X that helps. The X at m_3 joins m_1, m_2, m_7; the X at m_5 joins m_7, m_{13}, m_{15}; the X at m_{12} joins m_{13}, m_{14}, m_{15}.

Take the largest rectangles you can draw, then stop as soon as every 1 is covered.

$\therefore \boxed{F = A\overline{C}\,\overline{D} + \overline{A}D + \overline{A}C + AB}$

(The published answer to this past question is written as $F = A\overline{CD} + \overline{A}D + \overline{A}C + AB$; if your grouping produces an equivalent form with the same number of terms and literals, it is also correct — **K-map answers are not always unique**, and examiners accept any minimal form.)

3.6 Lesson 6 — the wrap-around, which everyone forgets

AB \ CD	00	01	11	10
00	1	0	0	1
01	0	0	0	0
11	0	0	0	0
10	1	0	0	1

Those four 1s are in the **four corners**. They look separated, but on a K-map the left edge touches the right edge and the top touches the bottom — the map is really a doughnut. So these four cells are one legal group of four.

Reading it: $B = 0$ throughout and $D = 0$ throughout; A and C change.

$\Rightarrow \boxed{F = \overline{B}\,\overline{D}}$

If you had grouped them as four separate single cells you would have written four 4-literal terms instead of one 2-literal term — and lost most of the marks.

3.7 Lesson 7 — full past questions, solved

Q1. Simplify $f(A,B,C) = \overline{ABC} + \overline{A}BC + A\overline{B}C + ABC$

[38th BCS]

The minterms are m_0, m_3, m_5, m_7.

A \ BC	00	01	11	10
0	1	0	1	0
1	0	1	1	0

Group 1 — m_3 and m_7 (column $BC = 11$, both rows): $B = 1, C = 1$, A changes $\Rightarrow BC$.
Group 2 — m_5 and m_7 (row $A = 1$, columns 01 and 11): $A = 1, C = 1$, B changes $\Rightarrow AC$.
Group 3 — m_0 alone: $A = 0, B = 0, C = 0 \Rightarrow \overline{A}\,\overline{B}\,\overline{C}$.

$\therefore \boxed{f = BC + AC + \overline{A}\,\overline{B}\,\overline{C}}$

Q2. Simplify using K-map: $F(W,X,Y,Z) = \Sigma(3,5,7,9,11,13,14,15)$

[Bank SO(AME) 2021]

WX \ YZ	00	01	11	10
00	0	0	1	0
01	0	1	1	0
11	0	1	1	1
10	0	1	1	0

Group 1 — the whole $YZ = 11$ column (m_3, m_7, m_{15}, m_{11}): $Y = 1, Z = 1 \Rightarrow YZ$.
Group 2 — m_5, m_7, m_{13}, m_{15} (columns 01 and 11, rows 01 and 11): $X = 1, Z = 1 \Rightarrow XZ$.
Group 3 — $m_9, m_{11}, m_{13}, m_{15}$ (columns 01 and 11, rows 11 and 10): $W = 1, Z = 1 \Rightarrow WZ$.
Group 4 — m_{14}, m_{15} (row $WX = 11$, columns 11 and 10): $W = 1, X = 1, Y = 1 \Rightarrow WXY$.

$\therefore \boxed{F = YZ + XZ + WZ + WXY}$

Q3. Simplify $F = \overline{A}BC + \overline{A}B\overline{C} + A\overline{B}\overline{C} + \overline{A}BC + ABC$ with logic circuit

[JBL SO(ANE) 2020]

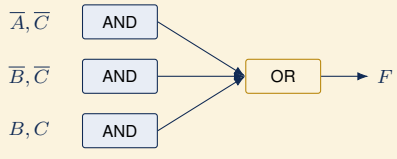
Minterms: m_0, m_2, m_3, m_4, m_7.

A \ BC	00	01	11	10
0	1	0	1	1
1	1	0	1	0

Group 1 — m_0, m_2 ($A = 0$, columns 00 and 10, i.e. $C = 0$): $\Rightarrow \overline{A}\,\overline{C}$.
Group 2 — m_0, m_4 (column $BC = 00$, both rows): $B = 0$, $C = 0 \Rightarrow \overline{B}\,\overline{C}$.
Group 3 — m_3, m_7 (column $BC = 11$, both rows): $B = 1$, $C = 1 \Rightarrow BC$.

$\therefore \boxed{F = \overline{A}\,\overline{C} + \overline{B}\,\overline{C} + BC}$

Now draw the circuit — this half of the question carries its own marks:



Q4. A comparator: $A = A_1A_0$, $B = B_1B_0$, output $F = 1$ when $A > B$

[BPSC SAE 2021]

There are 16 input combinations. $A > B$ is true for these ten: (01>00), (10>00), (10>01), (11>00), (11>01), (11>10) — as minterms of $A_1A_0B_1B_0$ these are $m_4, m_8, m_9, m_{12}, m_{13}, m_{14}$.

A_1A_0 \ B_1B_0	00	01	11	10
00	0	0	0	0
01	1	0	0	0
11	1	1	0	1
10	1	1	0	0

Group 1 — m_4, m_{12}: $A_0 = 1$, $B_1 = 0$, $B_0 = 0 \Rightarrow A_0\overline{B_1}\,\overline{B_0}$, which enlarges to $A_0\overline{B_1}$ when joined with m_{13}.
Group 2 — m_8, m_9, m_{12}, m_{13}: $A_1 = 1$, $B_1 = 0 \Rightarrow A_1\overline{B_1}$.
Group 3 — m_{12}, m_{14}: $A_1 = 1$, $A_0 = 1$, $B_0 = 0 \Rightarrow A_1A_0\overline{B_0}$.

$\therefore \boxed{F_{A>B} = A_1\overline{B_1} + A_0\overline{B_1}\,\overline{B_0} + A_1A_0\overline{B_0}}$

3.8 Lesson 8 — your self-check list

Before you write “therefore”, check these six things

1. Are my headings **00, 01, 11, 10**?
2. Is **every 1** inside at least one group?
3. Is every group a power of two, and rectangular?
4. Did I check the **wrap-around** — edges and corners?
5. Could any two of my groups be merged into one bigger group? (If yes, merge them.)
6. Did I use each **X** only where it made a group bigger, and never as a group on its own?

Then, if the question says “and draw the logic circuit”, **draw it**. One AND gate per term, one OR gate to collect them, inverters on the barred inputs.

PART 4

DIGITAL ELECTRONICS II — GATES & CIRCUITS

What they ask from this chapter

- What is a universal gate? Implement AND/OR/NOT/XOR using only NAND (or NOR) [45th BCS ICT 2024, BRiCM AME 2024, PGCB AE(CSE) 2024, MRA SCO 2023, BPSC AP 2021, SPCBL AME 2021, BMA AE 2021, BPSC ANE 2020]
- Draw the logic circuit of a given Boolean expression and find the output [Combined Bank AME/AE(IT) 2024, SB PLC ADBA 2024, BAPEX AGM(ICT) 2023]
- Half adder / full adder truth table and diagram [BPSC SAE 2021, PSC Lecturer 2017]
- Multiplexer, demultiplexer, encoder, decoder [PSC Lecturer 2017, BCSCL NE 2017]
- Flip-flops, counters, combinational vs sequential circuits

4.1 Logic gates

Digital circuits work with two voltage levels: **logic Low** written as **0**, and **logic High** written as **1**. A **logic gate** is a basic digital circuit with one or more inputs and a single output. Gates are the building blocks of every digital system.

The three categories — name them in this order

Basic gates: AND, OR, NOT. **Universal gates:** NAND, NOR. **Special gates:** Ex-OR, Ex-NOR.

Truth tables of all seven gates

A	B	AND $A{\cdot}B$	OR $A{+}B$	NAND $\overline{A{\cdot}B}$	NOR $\overline{A{+}B}$	Ex-OR $A{\oplus}B$	Ex-NOR $\overline{A{\oplus}B}$
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

The **NOT gate** has a single input and single output: $Y = \overline{A}$, so 0 becomes 1 and 1 becomes 0.

In one sentence each: AND gives 1 only when *all* inputs are 1. OR gives 1 when *any* input is 1. NAND is AND followed by an inverter. NOR is OR followed by an inverter. **Ex-OR gives 1 when the inputs are different**; $Y = A \oplus B = A\overline{B} + \overline{A}B$. **Ex-NOR gives 1 when the inputs are the same.**

4.2 Universal gates — the most repeated question in the syllabus

Why are NAND and NOR called universal gates?

Because **any other gate, and therefore any digital circuit whatsoever, can be built using only NAND gates — or only NOR gates**. NAND is logically an inverted AND; NOR is logically an inverted OR.
The advantage (they ask this separately): a manufacturer needs to produce and stock only *one* type of chip, which lowers cost, simplifies design and repair, and makes circuits easier to fabricate on an IC.

Building everything from NAND

Target gate	How many NANDs	Construction
NOT	1	Tie *both* inputs of a NAND to A. Output $= \overline{A \cdot A} = \overline{A}$.
AND	2	NAND the two inputs, then invert with a second NAND: $\overline{\overline{A \cdot B}} = A \cdot B$.
OR	3	Invert A and invert B (2 NANDs), then NAND the two results: $\overline{\overline{A} \cdot \overline{B}} = A + B$ by De Morgan.
NOR	4	Build OR with 3 NANDs, then invert with 1 more.
Ex-OR	4	The standard 4-NAND arrangement — and the question is often asked as "*maximum* 4 NAND gates".

Building everything from NOR

NOT	1	Connect all NOR inputs to A. Output $= \overline{A}$.
OR	2	NOR the inputs, then invert with a NOR inverter.
AND	3	Invert both inputs with NOR inverters, then NOR them: $\overline{\overline{A} + \overline{B}} = A \cdot B$ by De Morgan.

The method for "implement F using only NAND"

Step 1. Put F into **SOP** form (sum of products).
Step 2. Take the double complement of the whole thing: $F = \overline{\overline{F}}$.
Step 3. Apply De Morgan once to turn the OR of products into a NAND of NANDs.
Step 4. Draw it: one NAND per product term, one NAND to collect them, one NAND-inverter for each barred input.

Example: $F = AB + CD + E$ [45th BCS ICT 2024]
$F = \overline{\overline{AB + CD + E}} = \overline{\overline{AB} \cdot \overline{CD} \cdot \overline{E}}$
So: NAND(A,B) → first input; NAND(C,D) → second; NAND(E,E) → third; feed all three into a final NAND.
Total: 4 NAND gates.

4.3 Combinational circuits

Combinational vs sequential — write this as a table, they ask it directly

Combinational circuit	Sequential circuit
Output depends **only on the present input**	Output depends on the present input **and the past state**
No feedback path	**Feedback path is present**
Memory elements are not required	Memory elements play an essential role and are required
No clock needed	Usually clocked
Faster, easier to design	Slower, harder to design
Examples: adder, MUX, decoder, encoder	Examples: flip-flop, counter, register

Half adder and full adder

The **half adder** adds two single bits A and B. It produces a **Sum** S and a **Carry** C.

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Reading straight from the table:

$S = \overline{A}B + A\overline{B} = \boxed{A \oplus B}$

$C = \boxed{A \cdot B}$

So a half adder is **one Ex-OR gate and one AND gate**.

The **full adder** adds **three** bits: A, B and a carry C_{in} from the previous stage.

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

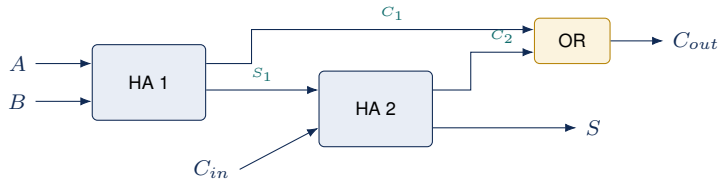
$$S = \overline{A}\,\overline{B}C_{in} + \overline{A}B\overline{C_{in}} + A\overline{B}\,\overline{C_{in}} + ABC_{in}$$
$$= \boxed{A \oplus B \oplus C_{in}}$$

$$C_{out} = \overline{A}BC_{in} + A\overline{B}C_{in} + AB\overline{C_{in}} + ABC_{in}$$
$$= \boxed{AB + BC_{in} + C_{in}A}$$

A full adder made from two half adders — the standard exam answer

First half adder: inputs A and $B \Rightarrow S_1 = A \oplus B,\ C_1 = AB$.
Second half adder: inputs S_1 and $C_{in} \Rightarrow S = S_1 \oplus C_{in} = A \oplus B \oplus C_{in},\ C_2 = S_1 C_{in}$.
One OR gate: $C_{out} = C_1 + C_2 = AB + (A \oplus B)C_{in}$.

So: **two half adders + one OR gate.** Say that sentence and draw the box diagram.



Multiplexer and demultiplexer

A **multiplexer (MUX)** selects **one of many inputs** and sends it to a single output — a "data selector". With n selection lines it handles 2^n inputs. A **demultiplexer (DEMUX)** does the reverse: one input, 2^n outputs.

4×1 MUX — data inputs I_0, I_1, I_2, I_3; selection lines S_1, S_0; output Y.

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

$$Y = \overline{S_1}\,\overline{S_0} I_0 + \overline{S_1} S_0 I_1 + S_1 \overline{S_0} I_2 + S_1 S_0 I_3$$

Implemented with 2 inverters, 4 AND gates and 1 OR gate.

Five applications of a multiplexer (asked directly): communication systems, telephone networks, computer memory addressing, transmission from computer systems of satellites, and data routing in arithmetic logic units.

Encoder and decoder

A **3-to-8 decoder** takes 3 inputs A, B, C and activates exactly one of 8 output lines:

$$D_0 = \overline{A}\,\overline{B}\,\overline{C} \qquad D_4 = A\overline{B}\,\overline{C}$$
$$D_1 = \overline{A}\,\overline{B}C \qquad D_5 = A\overline{B}C$$
$$D_2 = \overline{A}B\overline{C} \qquad D_6 = AB\overline{C}$$
$$D_3 = \overline{A}BC \qquad D_7 = ABC$$

Each output is one **minterm**. That is why a decoder plus OR gates can implement *any* Boolean function.

Parameter	Encoder	Decoder
Input applied	Active input signal (original message)	Coded binary input
Output generated	Coded binary output	Active output signal (original message)
Input lines	2^n	n
Output lines	n	2^n
Operation	Simple	Complex
Basic logic element	**OR gate**	**AND gate together with NOT gate**

4.4 Sequential circuits — latches, flip-flops, counters

Triggering

A clock signal has two **levels** (High, Low) and two **edges** (rising, falling).

Level triggering — the circuit responds while the clock is at a level. *Positive level triggering* means it operates while the clock is Logic High; *negative level triggering* while it is Logic Low.

Edge triggering — the circuit responds only at the instant the clock changes.

Latches are level-sensitive (controlled by an *enable* signal).	**Flip-flops are edge-sensitive** (controlled by a *clock* edge).

SR latch and SR flip-flop

An **SR latch** (Set–Reset) is built from two cross-coupled NOR gates with an enable E. It has inputs S and R and outputs $Q(t)$ and $\overline{Q(t)}$.

- If $S = 1$, the next state $Q(t{+}1)$ becomes **1** regardless of the present state.
- If $R = 1$, the next state $Q(t{+}1)$ becomes **0** regardless of the present state.
- If $S = R = 1$, the next state is **invalid / undefined** — this is the weakness of SR.

S	R	$Q(t)$	$Q(t{+}1)$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	**X**
1	1	1	**X**

Simplifying this table on a 3-variable K-map (with the two X cells used freely) gives the **characteristic equation**:

$$Q(t{+}1) = S + \overline{R}\,Q(t)$$

This is itself a common K-map question — it shows the two chapters connecting.

The four flip-flops

Type	Characteristic equation	Behaviour and use
SR	$Q(t{+}1) = S + \overline{R}Q(t)$	Set / Reset. $S{=}R{=}1$ is forbidden.
JK	$Q(t{+}1) = J\overline{Q} + \overline{K}Q$	The *modified* SR: the forbidden state now **toggles**. Obtained by feeding $S = J\overline{Q(t)}$ and $R = KQ(t)$.
D	$Q(t{+}1) = D$	*Data / Delay*. The output simply follows the input after one clock. Used in registers and for storing one bit.
T	$Q(t{+}1) = T \oplus Q(t)$	*Toggle*. $T{=}1$ flips the output, $T{=}0$ holds it. Obtained from a JK with J and K tied together. Used to build counters.

Race-around condition — a short but frequent question

In a JK flip-flop, when $J = K = 1$ and the clock pulse stays High longer than the propagation delay, the output toggles *repeatedly* within a single pulse. The final state becomes unpredictable. This is the **race-around condition**.

Cure: use a **master–slave** JK flip-flop (two flip-flops in series, clocked on opposite levels), or use edge triggering.

Counters and registers

A **counter** is a sequential circuit that counts clock pulses. With n flip-flops it counts from 0 to $2^n - 1$, i.e. it has 2^n states — so a 3-bit counter counts 0 to 7, a 4-bit counter 0 to 15.

Asynchronous (ripple) counter	Synchronous counter
The external clock is applied to **only the first** flip-flop	The external clock reaches **all** flip-flops at once
Each flip-flop's output clocks the next one	All flip-flops are clocked together
Slower — the change ripples through the chain	**Faster** — all stages change simultaneously
Simpler circuit, fewer gates	More complex, needs extra logic

A **register** is a group of flip-flops used to store a binary word. A **shift register** moves the stored bits left or right by one position on each clock pulse. Types by loading and reading method: **SISO** (serial-in serial-out), **SIPO**, **PISO**, **PIPO**.

PART 5

MICROPROCESSOR & ASSEMBLY LANGUAGE

What they ask from this chapter

- What is a microprocessor? Write its features / basic terms
- Features and block diagram of **8085**; features of **8086**
- **Addressing modes** of 8086 with examples — asked almost every year
- Difference between **microprocessor and microcontroller** — a guaranteed table question
- What is an interrupt? Types of interrupt; what is an ISR?
- Assembly language statement format; difference between **macro and procedure**
- What does an assembler do? Difference between **timer and counter**

5.1 What a microprocessor is

A **microprocessor** is the controlling unit of a micro-computer, fabricated on a small chip. It performs **ALU** (Arithmetic Logic Unit) operations and communicates with the other devices connected to it. In one line for the exam: *the microprocessor is the heart of the computer*.

Basic terms — learn these four definitions

Instruction set	The group of commands that the microprocessor can understand. It is the *interface between hardware and software*.
Bus	A set of conductors that carries data, address or control information. A microprocessor has **three** buses: **data bus, address bus, control bus**.
IPC	Instructions Per Cycle — how many instructions the CPU can execute in a single clock.
Clock speed	Operations per second, in MHz or GHz. Also called the **clock rate**.

How to remember the three buses

Address bus says *where* (one-way, CPU → memory). **Data bus** says *what* (two-way). **Control bus** says *when and how* (read? write?).

The address bus width decides how much memory can be addressed: 2^n **locations for** n **address lines**. 16 lines → $2^{16} = 64$ KB (8085). 20 lines → $2^{20} = 1$ MB (8086).

Features of a microprocessor (6-mark list question)

- **Low cost** — integrated circuit technology makes them cheap.
- **High speed** — executes millions of instructions per second.
- **Small size** — VLSI and ULSI integration reduces the whole computer's size.

- **Versatile** — the same chip can be used for many different applications.
- **Low power consumption** — uses metal oxide semiconductor (MOS) technology.
- **Less heat generation** — semiconductor technology, unlike vacuum tube devices.
- **Reliable** — very low failure rate, because it is made with semiconductor technology.
- **Portable** — small size and low power consumption make devices portable.

5.2 Classification of microprocessors

Based on data bus width	
8-bit	Early computers and embedded systems. *Intel 8080, Zilog Z80, Intel 8085*
16-bit	More complex calculations, better performance. *Intel 8086*
32-bit	Personal computers and workstations. *Intel 80386, ARM Cortex-A*
64-bit	High-performance and modern systems. *Intel Core i7, AMD Ryzen*
Based on microarchitecture	
Single-core	One processing core; executes one instruction at a time.
Multi-core	Several cores on one chip for parallel processing (dual, quad, hexa, octa).
Superscalar	Executes **more than one instruction per clock cycle** by dispatching instructions to multiple execution units.
Based on power and performance	
High-performance	Designed for speed. *Intel Xeon, AMD EPYC*
Low-power	Designed for energy efficiency, mobile/embedded. *Intel Atom, ARM Cortex-A7*
Based on special features	
Microcontroller	CPU + memory + I/O on a single chip. *Arduino, ESP32*
DSP	Digital Signal Processor, for signal processing. *TI TMS320*
FPGA-based	Field Programmable Gate Array, custom hardware. *Xilinx Zynq*

Superscalar — the exact answer they want

A **superscalar processor** is a CPU that implements a form of parallelism called **instruction-level parallelism** within a single processor. It can issue more than one instruction per clock cycle.

Pentium is superscalar; the 80386 is not, because the Pentium could issue more than one instruction per cycle and the 80386 could not. Note: each execution unit is *not* a separate processor — they are execution resources inside a single CPU, such as an ALU.

5.3 The Intel 8085 microprocessor

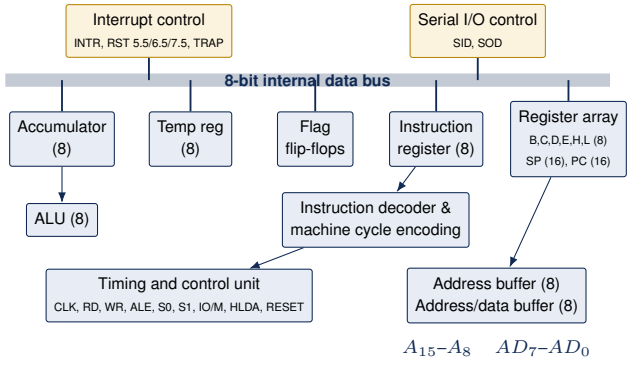
8085 is pronounced “eighty-eighty-five”. It is an **8-bit** microprocessor designed by **Intel in 1977** using **NMOS** technology.

8085 configuration — memorise these six lines

- **8-bit** data bus
- **16-bit** address bus, which can address up to **64 KB**
- A **16-bit** program counter (PC)
- A **16-bit** stack pointer (SP)
- **Six 8-bit registers** arranged in pairs: **BC, DE, HL**
- Requires a **+5 V** supply and operates at a **3.2 MHz** single-phase clock

Block diagram of the 8085 — what to draw

If they ask for the block diagram, draw these blocks and connect them all to the **8-bit internal data bus**:



8085 interrupts — in priority order

Interrupt	Type	Note
TRAP	Non-maskable	**Highest priority**; cannot be disabled
RST 7.5	Maskable	Edge triggered
RST 6.5	Maskable	Level triggered
RST 5.5	Maskable	Level triggered
INTR	Maskable	**Lowest priority**; general purpose

5.4 The Intel 8086 microprocessor

8086 features — the standard answer

- It has an **instruction queue** capable of storing **six instruction bytes** from memory, which results in faster processing (this is pipelining).
- It was the **first 16-bit processor**: 16-bit ALU, 16-bit registers, 16-bit internal data bus and 16-bit external data bus.
- **20-bit address bus** $\Rightarrow$ can address **1 MB** of memory.
- Available in three versions by frequency: **8086** $\rightarrow$ **5 MHz**, **8086-2** $\rightarrow$ **8 MHz**, **8086-1** $\rightarrow$ **10 MHz**.
- Divided into two independent units: the **BIU** (Bus Interface Unit — fetches instructions, handles addresses) and the **EU** (Execution Unit — decodes and executes).

5.5 Addressing modes of the 8086

Definition to write: the way in which an operand is specified for an instruction — in the accumulator, in a general purpose register, or in a memory location — is called the **addressing mode**. The 8086 has **8 addressing modes**.

Mode	Where the operand is	Example
Register	In an 8-bit or 16-bit general purpose register	`MOV AX, CX` `ADD AL, BL`
Immediate	In the instruction itself	`MOV AL, 35H` `ADD AX, 4836H`
Direct	Effective address written directly in the instruction	`MOV AX, [1592H]` `MOV AL, [0300H]`
Register indirect	At the memory offset held in BP, BX, DI or SI	`MOV AX, [BX]`
Based	Offset = contents of BX/BP + displacement	`MOV DX, [BX+04]`
Indexed	Offset = contents of SI/DI + displacement	`MOV BX, [SI+16]`
Based-index	Offset = base register + index register	`MOV AX, [BX+SI]`
Relative based-index	Offset = base + index + displacement	`MOV AX, [BX+SI+06]`

The two-second way to identify a mode in an MCQ

Look at the *second* operand.
No brackets, a register → **register**. No brackets, a number → **immediate**.
Brackets with a plain number → **direct**. Brackets with one register → **register indirect**.
Brackets with BX/BP + number → **based**. Brackets with SI/DI + number → **indexed**.
Brackets with two registers → **based-index**.

5.6 Instruction set — the groups

Data transfer	`MOV` (move), `PUSH/POP` (stack), `XCHG` (exchange), `IN/OUT` (port), `LEA` (load effective address)
Arithmetic	`ADD`, `ADC` (add with carry), `SUB`, `SBB`, `INC`, `DEC`, `MUL`, `DIV`, `CMP` (compare)
Logical	`AND`, `OR`, `XOR`, `NOT`, `TEST`
Rotate/shift	`ROL`, `ROR` (rotate left/right), `RCL`, `RCR` (rotate through carry), `SHL`, `SHR`
Branch	`JMP`, `JZ/JNZ`, `JC/JNC`, `CALL`, `RET`, `LOOP`
Machine control	`HLT`, `NOP`, `WAIT`, `ESC`

Definitions they ask for individually

OR — performs a bit-by-bit logical OR of two operands and places the result in the specified destination. **XOR** — bit-by-bit logical XOR. **NOT** — takes the **1's complement** of the contents of a specified register or memory location. **TEST** — performs a logical AND of two operands but only sets the flags; it does *not* store the result.

> **RCL/RCR** — rotate all bits left/right *through the carry flag*. **ROL/ROR** — rotate all bits left/right by the specified number of bits.

5.7 Interrupts

An **interrupt** is a process of creating a temporary halt during program execution, which allows peripheral devices to access the microprocessor. The microprocessor responds with an **Interrupt Service Routine (ISR)** — a short program or subroutine that instructs the microprocessor how to handle the interrupt.

Hardware interrupt	Caused by an external device through a physical pin (e.g. keyboard press, TRAP, INTR).
Software interrupt	Caused by an instruction inside the program itself (e.g. `INT n`).
Maskable	Can be disabled/ignored by the programmer (INTR, RST 5.5/6.5/7.5).
Non-maskable	Cannot be disabled — always serviced (TRAP). Used for critical events like power failure.

5.8 Assembly language

Assembly language is a low-level language that uses **mnemonics** (short symbolic names) instead of binary machine code. It is machine-dependent — code written for one processor family will not run on another.

Format of an assembly statement

An assembly language statement has up to four parts:

1. Label	An identifier followed by a colon `(:)`. It names a memory address so it can be jumped to.
2. Mnemonic	The operation to perform, e.g. `MOV`, `ADD`, `JMP`.
3. Operand(s)	The data the instruction works on — registers, memory, or immediate values.
4. Comment	Begins with a **semicolon** `(;)`. May contain any printable character. It can be on a line by itself or after an instruction.

```
START:  MOV  AX, BX      ; move BX into AX
        ADD  AX, 05H      ; add 5 to AX
        HLT               ; stop
; label    mnemonic  operands     comment
```

Advantages and disadvantages of assembly language

Advantages: directly controls hardware; needs very little memory and execution time; suitable for time-critical jobs; ideal for writing **interrupt service routines** and memory-resident programs; makes complex hardware-specific jobs easier.

Disadvantages: machine dependent (not portable); long programs are hard to write, read and maintain; the programmer must know the hardware in detail.

An assembler

An **assembler** is a program that converts assembly language into **machine code**. It takes the basic commands and operations from assembly code and converts them into binary code that a specific type of processor can recognise. Assemblers are similar to compilers in that they produce executable code.

5.9 Two difference tables that come up repeatedly

Microprocessor vs Microcontroller

Microprocessor	Microcontroller
Used for big applications	Used to execute a single task within an application
It is the heart of a computer system	It is the heart of an embedded system
Just a processor — memory and I/O must be connected **externally**	Contains the processor along with **internal** memory and I/O components
Cost of the entire system increases	Cost of the entire system is low
Power consumption is high	Power consumption is low
Most have no power-saving mode	Most have a power-saving mode
Difficult to replace	Easy to replace
Mainly used in personal computers	Used in washing machines, MP3 players, etc.

Macro vs Procedure

Macro	Procedure
A sequence of instructions written inside the macro definition to support modular programming	A set of instructions that can be called repetitively to perform a specific task
Requires **more** memory	Requires **less** memory
Does **not** require CALL and RET instructions	**Requires** CALL and RET instructions
Machine code is generated **each time** the macro is called	Machine code is generated **only once**
Used for **fewer than 10** instructions	Used for **more than 10** instructions

Timer vs Counter

Counter	Timer
The register increments counting 1-to-0 transitions at an **external** input pin (T0, T1)	The register increments for **every machine cycle**
Uses an **external signal** to count pulses	Uses the **internal clock** frequency and generates a delay
Maximum count rate is **1/24** of the oscillator frequency	Maximum count rate is **1/12** of the oscillator frequency

AVR microcontroller — one short paragraph is enough

The **AVR** microcontroller is an advanced version of the microprocessor. It contains an on-chip CPU, ROM, RAM, I/O unit and interrupt controller. Because everything is on one chip, a microcontroller is used for

> high-speed, dedicated control applications.

PART 6

SOFTWARE ENGINEERING & SDLC

What they ask from this chapter

- Write down the **SDLC steps** with explanation — the single most repeated question [BRiCM AME 2024, EGCB SAE(ICT) 2023, DESCO AE 2023, NPCBL ET(Soft) 2023, BTRC SAD 2021, BPSC SAE 2021, 40th BCS 2020, BPSC ANE 2020]
- Characteristics of software; define **black box** and **white box** testing [45th BCS ICT 2024]
- Difference between the models (Waterfall vs Agile, Agile vs RAD) [PBL SQA 2023, BPSC AP 2022]
- What is **SRS**? At which stage of development is it made? [BPSC AP, 40th BCS 2021]
- **SQA** — what will you look for in quality software? [Combined Bank SO(IT) 2024]
- Responsibilities of a software engineer [45th BCS CSE 2024]
- Risk analysis techniques [38th BCS]; **SMLC** steps [BUET MSc Oct 2019]
- Cohesion vs coupling; verification vs validation; buy-vs-build calculation [NWPGCL AM(ICT) 2024]

6.1 Software engineering basics

Software engineering is the process of analysing user requirements and then designing, building and testing a software application that satisfies those requirements.

Characteristics of good software — remember "OTM"

Operational	How well the software works in operation: budget, usability, efficiency, correctness, functionality, dependability, security, safety.
Transitional	How well it moves from one environment to another: portability, interoperability, reusability, adaptability.
Maintenance	How well it can be maintained in a changing environment: modularity, maintainability, flexibility, scalability.

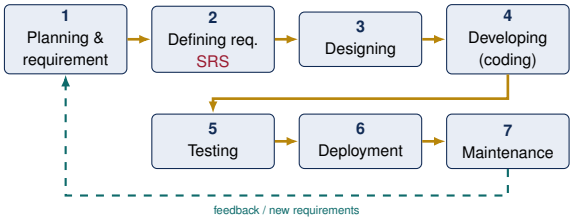
The four key software process activities

- **Software specification** — the functionality of the software and the constraints on its operation must be defined.
- **Software development** — the software that meets the requirement must be produced.
- **Software validation** — the software must be validated to ensure it does what the customer wants.
- **Software evolution** — the software must evolve to meet changing client needs.

6.2 SDLC — the Software Development Life Cycle

A **software life cycle model** (also called a process model) is a pictorial and diagrammatic representation of the software life cycle. It represents all the methods required to make a software product transit

through its life cycle stages.



1. Planning & requirement analysis	The **most important** stage. Performed by senior team members with input from all **stakeholders** and domain experts (SMEs). Quality assurance planning and identification of project **risks** also happen here.
2. Defining requirements	The requirements are documented and accepted by the stakeholders. This is done through the **SRS — Software Requirement Specification** document, which contains all the product requirements to be built during the life cycle.
3. Designing the software	Brings together all the knowledge of requirements and analysis. This phase is the product of the previous two.
4. Developing the project	Actual development and programming begins. Developers follow the coding guidelines and use compilers, interpreters and debuggers.
5. Testing	The code is tested against the requirements. **Unit, integration, system and acceptance testing** are done in this stage.
6. Deployment	Once certified and free of stated bugs, the software is deployed and released.
7. Maintenance	When clients start using the system, real issues appear and are solved from time to time. This care of the developed product is maintenance.

Two traps in this question

1. If asked *"which stage ensures user acceptance?"* the answer is the **testing stage**, specifically **acceptance testing** — not deployment.
2. If asked *"at which stage is the SRS made?"* the answer is **stage 2, defining requirements** — not stage 1.

SMLC — Software Maintenance Life Cycle

SMLC is the process of modifying software after delivery to correct faults, improve performance and adapt it to a changed environment. Its steps:

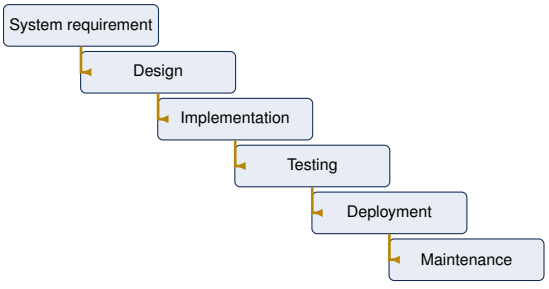
1. **Identification and tracing** — identify the requirement for modification.
2. **Analysis** — analyse the impact and cost of the change.
3. **Design** — design the new or changed modules.
4. **Implementation** — write the new code.
5. **System testing** — integration testing of the changed modules.
6. **Acceptance testing** — accepted by the users.
7. **Delivery** — deliver the modified system to the customer.

Types of maintenance: Corrective (fix faults), **Adaptive** (adapt to a new environment), **Perfective** (improve performance or add features), **Preventive** (reduce future problems).

6.3 The SDLC models

Waterfall model

Development flows steadily downwards through the phases like a waterfall. A phase must be complete before the next one begins.



Advantages: simple to implement and needs minimal resources; the start and end point of each phase is fixed, which makes progress easy to track; easy control and clarity for the customer due to a strict reporting system.

Disadvantages: the risk factor is higher, so it is **not suitable for large and complex projects**; it **cannot accept changes** in requirements during development; working software is produced only late in the life cycle.

V-model (verification and validation model)

Each development phase on the left has a directly matching testing phase on the right. Requirement analysis pairs with acceptance testing, system design with system testing, architectural design with integration testing, and module design with unit testing.

Advantage: testing is planned early, so defects are found early. Simple and easy to understand. **Disadvantage:** very rigid and least flexible; no early prototypes are produced; not good for complex or changing projects.

Incremental / iterative model

The requirement is broken into multiple builds. Each build (increment) goes through its own requirement, design, coding and testing cycle, and modules are added until the full system is ready.

Advantage: working software is produced early; easier to test and debug a small iteration; flexible to changes. **Disadvantage:** needs good planning and design; the total cost is higher than waterfall.

Spiral model

The spiral model combines iterative development with **risk analysis**. Each loop of the spiral has four quadrants:

1. Objective setting	Identify objectives, alternatives and constraints for this cycle.
2. Risk assessment	Identify and resolve the risks — **the defining feature of this model**.
3. Development & validation	Develop and test the product of this cycle.
4. Planning	Review the result and plan the next loop of the spiral.

Advantage: best for **large, expensive, high-risk** projects; risk is handled explicitly. **Disadvantage:** costly; requires expertise in risk analysis; not suitable for small projects.

RAD — Rapid Application Development

The central theme of RAD is designing **quick prototypes** which are then refined into production-quality code. It relies heavily on reusable components and a strong, experienced team.

Prototype model

A working sample (prototype) with limited functionality is built first and shown to the customer. Their feedback refines the requirements, and only then is the real product built. Good when requirements are unclear at the start.

Agile model

Agile promotes continuous interaction between development and testing throughout the SDLC. The project is divided into small incremental builds; all builds are provided in iterations, and **each iteration lasts from one to three weeks**.

The six phases of Agile — write them in this order

Requirements gathering → Design the requirements → Construction/iteration → Testing/Quality assurance → Deployment → Feedback.

Advantages: promotes teamwork and cross-training; functionality can be developed rapidly and demonstrated; resource requirements are minimum; suitable for fixed *or* changing requirements; a good model for environments that change steadily.

Disadvantages: not suitable for very small projects; needs an expert to take decisions; the cost of Agile is a little more than other models; documentation is minimal, so if a key person leaves, knowledge is lost.

Agile testing methods (name at least four): Scrum, **Crystal**, **DSDM** (Dynamic Software Development Method), **FDD** (Feature Driven Development), **Lean Software Development**, **XP** (eXtreme Programming).

Agile vs Waterfall — the table they want

Agile model	Waterfall model
Follows an incremental approach	Follows a sequential design process
Allows requirements to change **after** development starts, so it is more flexible	Rigid — does not allow requirements to change after development starts
Testing is done **simultaneously** with development	Testing is done **after** the build step
Known for its versatility	A systematic, sometimes rigorous, developing method
Small projects can be delivered very quickly	Not suitable if requirements are unclear
Customer involvement is continuous	Customer is mainly involved at the beginning and the end

Agile vs RAD

Agile model	RAD model
Does not recommend developing prototypes; emphasises systematic development of each incremental feature	Central theme is designing **quick and dirty prototypes** which are then refined into production-quality code
Breaks the solution into features that are incrementally developed and delivered	Developers build all the features first (badly), then successively improve the code
Not suitable for small projects that cannot be divided	Hard to use if the company has not developed a similar project before, as there is no code to reuse

6.4 Requirement engineering

Requirement engineering (RE) is the process of defining, documenting and maintaining requirements in the engineering design process. It provides the mechanism to understand what the customer desires, analyse the need, assess feasibility, negotiate a reasonable solution, specify the solution clearly, validate the specification, and manage the requirements as they are transformed into a working system.

The five steps of the requirement engineering process

Feasibility study → Requirement elicitation and analysis → Software Requirement Specification → Software requirement validation → Software requirement management.

Feasibility study — three types

Technical	Assesses whether the current resources and technology are enough to meet the user's requirements within the allocated budget and time.
Operational (behavioural)	Checks whether the human beings or employees in the business will actually use it, and whether the software will operate properly after it is developed.
Economic	Determines whether the software has the potential to generate financial gain — covering development cost, cost of the study, and estimated software and hardware cost.

SRS — Software Requirement Specification

The **SRS** is a document that describes *what* the software will do and how it is expected to perform. It is created in **stage 2** of the SDLC, after requirement analysis. A good SRS is **correct, complete, consistent, unambiguous, verifiable, modifiable and traceable**.

Risk analysis techniques

Risk identification (checklists, brainstorming, past project data) → **Risk analysis** (estimate probability and impact of each risk) → **Risk prioritisation** (rank by risk exposure = probability × loss) → **Risk management planning** (avoidance, transfer, mitigation, contingency plan) → **Risk monitoring**.

Categories: **project risks** (schedule, budget, staff), **technical risks** (design, implementation, interfacing), and **business risks** (market, sales, management).

6.5 Software design concepts

Modularity

Modularity is the division of software into separate modules which are differently named and addressed, and are integrated later to obtain the complete functional software. It is the only property that allows a program to be intellectually manageable.

Advantages: large programs can be written by several people; encourages commonly used routines to be placed in a library and reused; simplifies loading a large program into main storage; provides a framework for complete testing and is more accessible to test; produces a well-designed, more readable program.

Disadvantages: execution time may be longer; storage size may be increased; compilation and loading time may be longer; inter-module communication problems may increase.

Cohesion vs coupling — know the direction

Cohesion	Coupling
Measures the relative functional **strength within** a module	Measures the relative **interdependence between** modules
It is an **intra**-module concept	It is an **inter**-module concept
We want cohesion to be **HIGH**	We want coupling to be **LOW**
Types (best to worst): functional, sequential, communicational, procedural, temporal, logical, coincidental	Types (best to worst): data, stamp, control, external, common, content

One-line memory hook

High cohesion, low coupling. A module should do *one job well* (cohesion) and *depend on others as little as possible* (coupling).

Information hiding means each module hides its internal design decisions from other modules; they communicate only through well-defined interfaces. **Functional independence** is achieved through high cohesion and low coupling.

6.6 Software testing

Software testing is the process of identifying the correctness of software by considering all its attributes (reliability, scalability, portability, re-usability, usability) and evaluating the execution of software components to find bugs, errors or defects.

White box vs black box vs grey box

Black box	White box	Grey box
Knowledge of the internal code is **not required**; only the GUI is needed for test cases	Knowledge of the internal code is **necessarily required**	**Partial** knowledge of the internal structure is required
Performed by the **test engineer**	Performed by the **developer**	Tester, developer and end user can all take part
Not considered for algorithm testing	**Well suited** for algorithm testing	Not considered for algorithm testing
The **least** time-consuming process	The **most** time-consuming process	Less time-consuming than white box
Also called functional / behavioural testing	Also called structural / glass box testing	Combination of both

Advantages of white box: optimises code so hidden errors are found; test cases can be automated easily; more thorough because it covers all code paths; can begin in the SDLC phase even without a GUI.

Disadvantages of white box: very time consuming for large applications; expensive and complex; can lead to production errors because it is not detailed by the developers; needs professional programmers with detailed knowledge of the language and implementation.

Levels of testing

Unit testing	Each individual module or component is tested on its own. Usually white box, done by the developer.
Integration testing	Modules are combined and tested as a group to find interface defects.
System testing	The complete, integrated system is tested against the requirements.
Acceptance testing	The customer/user tests whether the system meets the business need. This is the stage that **ensures user acceptance**.

Alpha vs beta testing

Alpha testing	Beta testing
Done **in-house** by internal testers	Done by **real users** in their own environment
Uses **both** white box and black box techniques	Uses **only black box** testing
Long execution cycles may be required	Only a few weeks are required
Performed **before** the product launch	Performed at the time of product marketing

Verification vs validation

Verification	Validation
"Are we developing the **right product**?" — checking against the specification	"Is the **developed product right**?" — checking against the user's actual need
Also known as **static** testing	Also known as **dynamic** testing
Quality assurance (QA) comes under verification	**Quality control (QC)** comes under validation
Code is **not executed**	Code **is executed**
Done **before** validation	Done **after** verification
Reviews, walkthroughs, inspections	Unit, integration, system, acceptance testing

6.7 Software quality assurance (SQA)

SQA is a set of planned and systematic activities that ensure the software process and product conform to requirements and standards. It is **process-oriented** and runs throughout the whole SDLC, unlike quality control which is **product-oriented** and checks the finished item.

"As an SQA team leader, what will you look for when buying software?"

Answer with these attributes: **Functionality** (does it do the required job), **Reliability**, **Usability**, **Efficiency/performance**, **Maintainability**, **Portability**, **Security**, **Scalability**, **Vendor support and documentation**, **Compliance with standards**, and **total cost of ownership** including licence and maintenance.

Six Sigma

Six Sigma is the process of improving quality by removing defects and their causes. The quality standard is defined as **3.4 defects per million** opportunities. Its two methodologies:

DMAIC — **D**efine, **M**easure, **A**nalyse, **I**mprove, **C**ontrol. Used to *improve an existing* product or process.
DMADV — **D**efine, **M**easure, **A**nalyse, **D**esign, **V**erify. Used to *create new* product or process designs.

6.8 Responsibilities of a software engineer

Analyse and gather user requirements; design the system architecture and modules; write clean, efficient and maintainable code; test and debug the software; document the design and the code; ensure security, performance and scalability; maintain and update the software after delivery; collaborate with stakeholders and the team; follow standards, ethics and the agreed process model.

6.9 Design patterns (short notes)

Creational — concerned with how objects are created (e.g. **Singleton**, Factory).
Structural — concerned with class and object composition; uses inheritance to compose interfaces (e.g. Adapter, Decorator).
Behavioural — concerned with communication between objects (e.g. Observer, Strategy).
J2EE patterns — concerned with the presentation tier, identified by Sun Java Center.

Singleton pattern: defines a class that has **only one instance** and provides a global point of access to it. Two forms: **early instantiation** (created at load time) and **lazy instantiation** (created when required). **Advantage:** saves memory, because the object is not created at each request — a single instance is reused. **Used in** logging, caching, thread pools, configuration settings, and database applications.

The buy vs build question [NWPGCL AM(ICT) 2024]

When asked to choose between **Buy**, **Build**, or **Open source + modification**, compute the **expected cost** of each option and pick the lowest, then justify with non-cost factors.

Buy $= 50$ lac (certain).
Build $= 0.30 \times 40 + 0.70 \times 50 = 12 + 35 =$ **47** lac.
Open source $= 0.20 \times 30 + 0.80 \times 50 = 6 + 40 =$ **46** lac.

Lowest expected cost is **open source with modification**. Then add the qualitative reasoning the examiner wants: buying gives fastest delivery but no control over the source; building gives full control but the highest risk and time; open source balances cost and control but depends on licence terms and community support.

PART 7

RAPID-FIRE QUESTION BANK

How to use these four pages

Cover the right-hand column with a sheet of paper. Say the answer out loud, then slide the paper down. Do one section a day. Anything you miss twice, go back and re-read that section of the book — do not just memorise the line.

7.1 Linux

Question	Answer
Who created Linux, when and where?	Linus Torvalds, April 1991, University of Helsinki, Finland, aged 21.
Linux vs Unix in one line?	Linux is just the **kernel** and is open source (GPL); Unix is a complete OS and is copyrighted.
Layers of Linux architecture?	Hardware → Kernel → Shell → Applications → User.
Three ways Linux provides security?	Authentication, authorization, encryption.
Show last 10 lines of a continuously updating log [Titas Gas AE(CSE) 2024]	`tail -f logfile`
Show hidden files and directories [Milk Vita AM(CSE) 2023]	`ls -a` (or `ls -la` for details)
Delete a directory and its files	`rm -r dirname` (`rmdir` only removes empty ones)
Print the last five lines of a text file	`tail -n 5 filename.txt`
Download a file from a URL	`wget http://example.com/file.txt`
Show the 11th to 15th line of `myfile` [BTCL AME 2023]	`sed -n '11,15p' myfile`
Show permission for read/write/execute on `myfile`	`chmod 777 myfile` (or `ls -l myfile` to view)
Show the current file directory	`pwd`
Make a directory named PSC [BPSC SAE 2021]	`mkdir PSC`
Copy a directory with all its contents into `/home/admin` [BPSC SAE 2021]	`cp -r PSC /home/admin`
Create a user [PGCB SAE(CSE) 2021]	`useradd username` then `passwd username`
Show all files including hidden ones inside the home directory [NESCO AM(ICT) 2021]	`ls -a ~`
Show disk space in human-readable format [NESCO AM(ICT) 2021]	`df -h`
Find previous commands containing a string [BCC AP 2021]	`history | grep pattern`
Create file `apscl.txt`, give all permissions, read first 7 lines, delete it [APSCL AE(ICT) 2021]	`touch apscl.txt` · `chmod 777 apscl.txt` · `head -n 7 apscl.txt` · `rm apscl.txt`

Question	Answer
Ownership and group permission command [PBL HE 2023]	`chown user:group filename` and `chgrp group filename`
What is RPM? What is YUM?	RPM = Red Hat Package Manager (build/install/upgrade/query/remove packages). YUM = Yellowdog Updater Modified, an open-source utility that manages RPM packages.
GPT vs MBR?	GPT: >2 TB disks, 128 primary partitions, GUIDs, CRC32 integrity, part of UEFI. MBR: max 2 TB, only 4 primary partitions.

7.2 Number systems, codes and Boolean algebra

Question	Answer
$(E)_{16}$ in decimal, octal and binary [45th BCS CSE 2024]	$E = 14_{10} = 16_8 = 1110_2$
Next four after 6FC, 6FE [45th BCS CSE 2024]	Step is 2: **700, 702, 704, 706**
2's complement of $(+55)_{10}$ [45th BCS CSE 2024]	$55 = 00110111$; 1's $= 11001000$; 2's $=$ **11001001**
$(423)_{10}$ to octal and $(3000)_{10}$ to hex [BPDB AE(CSE) 2024]	$(423)_{10} = (647)_8$; $(3000)_{10} = (BB8)_{16}$
$(651)_8$ to decimal and hex [BPSC AME 2021]	$= 6{\cdot}64 + 5{\cdot}8 + 1 =$ **425_{10}**; binary $110\,101\,001 \rightarrow$ **$1A9_{16}$**
$(1AC)_{16}$ to binary and decimal [DPDC 2020]	**$1\,1010\,1100_2$**; $256 + 160 + 12 =$ **428_{10}**
$(2345)_{10}$ to hex [ICT Ministry AP 2017]	**929_{16}**
$(ABCD)_{16}$ to octal [ICT Ministry AP 2017]	Binary $1010\,1011\,1100\,1101 \rightarrow$ group in 3s $\rightarrow$ **125715_8**
$(CDAB)_{16}$ to octal [BTCL JAM 2024]	$1100\,1101\,1010\,1011 \rightarrow$ **146653_8**
What is 2's complement and why is it used?	1's complement $+1$. It lets the computer perform **subtraction using addition**, and gives a single representation of zero.
$(-24)_{10} - (+11)_{10}$ using 2's complement [18th NTRCA 2024]	$-24 - 11 = -35$. $35 = 00100011 \rightarrow$ 2's $=$ **11011101**
BCD addition $00010011 + 00100110$ [NPCNL JAM 2022]	$13 + 26 = 39$. Both groups ≤ 9, so no correction: **0011 1001**
Excess-3 code of 1010 [DPDC 2020]	1010 is decimal 10 $\rightarrow$ digits 1 and 0 $\rightarrow$ Excess-3 codes 0100 and 0011 $\rightarrow$ **0100 0011**
Difference between BCD and binary code?	Binary converts the **whole number** at once; BCD converts **each decimal digit** separately into 4 bits. BCD wastes codes 1010–1111.
Parity method for error detection [BB AP 2023]	An extra bit makes the total number of 1s even (or odd). The receiver recounts; a wrong count means an error. Detects single-bit errors only, cannot correct them.
Bit string of "Delta" in ASCII [BB AP 2023]	Hex 44 65 6C 74 61 $\rightarrow$ **01000100 01100101 01101100 01110100 01100001**
State De Morgan's law [BPSC AP 2023]	$\overline{A+B} = \bar{A}\cdot\bar{B}$ and $\overline{A\cdot B} = \bar{A}+\bar{B}$. In words: break the bar, change the sign.
Simplify $F = A'B'C + A'BC + AB'$ [45th BCS CSE 2024]	$A'C(B'+B) + AB' =$ **$\mathbf{A'C + AB'}$**
Simplify $Y = A\bar{B} + (A+B)C$ [BPSC ADA 2022]	$A\bar{B} + AC + BC =$ **$\mathbf{A\bar{B} + AC + BC}$**; with consensus, $= A\bar{B} + C(A+B)$
Simplify $(x + y + xy)(x + z)$ [BPSC AME 2021]	$x + y + xy = x + y$; so $(x+y)(x+z) =$ **$\mathbf{x + yz}$**
Simplify $\bar{x}\bar{y}z + \bar{x}yz + x\bar{y}$ [BPSC AP 2021]	$\bar{x}z(\bar{y}+y) + x\bar{y} =$ **$\mathbf{\bar{x}z + x\bar{y}}$**

7.3 Gates, K-map and circuits

Question	Answer
What is a universal gate and why? [BRiCM AME 2024, NPCNL JAM 2022]	NAND and NOR. Any other gate (NOT, AND, OR, XOR) — and therefore any digital circuit — can be built using only that one gate type.
NOT using NAND / NOR?	Tie both inputs together to the same signal.
AND using only NAND?	NAND followed by a NAND-inverter (2 NANDs).
OR using only NAND?	Invert both inputs with NANDs, then feed a NAND (3 NANDs).
AND using only NOR?	Invert both inputs with NORs, then a NOR (3 NORs).
OR using only NOR?	NOR followed by a NOR-inverter (2 NORs).
XOR using NAND, maximum 4 gates [PGCB AE(CSE) 2021]	4 NAND gates: $Y = \overline{A \cdot \overline{AB}} \cdot \overline{B \cdot \overline{AB}}$
$\sum(3, 5, 7, 9, 11, 13, 15)$ simplified [Bank SO(AME) 2021]	All odd minterms $\Rightarrow$ **Z** is 1 in every one $\Rightarrow$ $F =$ **Z** (for W, X, Y, Z with Z as LSB).
$F = \overline{A}BC + A\overline{BC} + A\overline{B}C + \overline{A}B\overline{C} + ABC$ by K-map [38th BCS]	Group: $\mathbf{F = \overline{A}B + A\overline{B} + BC}$ (check each group by dropping the changing variable).
Rule for reading a K-map group?	Drop every variable that **changes** inside the group; keep the ones that stay constant. $0 \rightarrow$ barred, $1 \rightarrow$ unbarred (for SOP).
Why is the order 00, 01, 11, 10?	So that **only one bit changes** between adjacent cells (Gray code) — that is what makes grouping valid.
What is a don't-care condition?	An input combination that can never occur or whose output does not matter. Marked $\times$. Use it as **1** to make a bigger group, or ignore it.
Half adder equations?	$S = A \oplus B,\ C = A \cdot B$
Full adder equations?	$S = A \oplus B \oplus C_{in},\ C_{out} = AB + BC_{in} + AC_{in}$
A full adder is made of? [PSC Lecturer 2017]	**Two half adders and one OR gate.**
Functionality of a multiplexer [PSC Lecturer 2017]	Selects one of 2^n data inputs and routes it to a single output, based on n select lines. "Many to one."
Five applications of a multiplexer [BCSCL NE 2017]	Data routing/selection, communication systems, parallel-to-serial conversion, waveform generation, implementing Boolean functions (logic function generator).
Encoder vs decoder?	Encoder: 2^n inputs $\rightarrow$ n outputs, built from **OR** gates, simple. Decoder: n inputs $\rightarrow$ 2^n outputs, built from **AND** + **NOT** gates, complex.
Latch vs flip-flop?	Latch is **level**-triggered (works while enable is high); flip-flop is **edge**-triggered (works on a clock transition).
Which flip-flop has an invalid state, and which cures it?	**SR** is invalid at $S = R = 1$; **JK** cures it by making that combination **toggle**. **D** has no invalid state.
Race-around condition?	In a JK flip-flop with $J = K = 1$, the output toggles repeatedly while the clock is high. Cured by **master–slave** or edge triggering.
Combinational vs sequential circuit?	Combinational: no memory, no feedback, output depends only on present input (adder, MUX). Sequential: has memory and feedback, output depends on present input **and** past state (flip-flop, counter).
n flip-flops count up to?	0 to $2^n - 1$, i.e. mod-2^n. A mod-10 counter needs **4** flip-flops.
Synchronous vs asynchronous counter?	Synchronous: same clock to all flip-flops, fast, no cumulative delay. Asynchronous (ripple): output of one clocks the next, simpler but slower.

7.4 Microprocessor, assembly and software engineering

Question	Answer
8085 — year, bits, buses, memory?	Intel 1977, 8-bit data bus, 16-bit address bus, addresses **64 KB**, +5 V, 3.2 MHz.
8085 register pairs?	BC, DE, HL (six 8-bit registers), plus accumulator, flag register, 16-bit PC and 16-bit SP.
Highest priority 8085 interrupt?	**TRAP** — and it is non-maskable.

Question	Answer
8086 — bits, address bus, memory, queue?	First 16-bit processor; 20-bit address bus → **1 MB**; **6-byte** instruction queue; BIU and EU.
How many addressing modes does the 8086 have?	**Eight.**
`MOV AL, 35H` is which mode?	**Immediate** — the operand is inside the instruction.
`MOV AX, [BX]` is which mode?	**Register indirect.**
`MOV DX, [BX+04]` is which mode?	**Based** addressing.
What is an interrupt and an ISR?	An interrupt temporarily halts execution so a peripheral can access the processor; the ISR is the short routine that tells the processor how to handle it.
Maskable vs non-maskable?	Maskable can be disabled by the programmer (INTR, RST 5.5/6.5/7.5); non-maskable cannot (TRAP).
Microprocessor vs microcontroller?	μP: only the processor, external memory/IO, costly, high power, used in PCs. μC: CPU + memory + I/O on one chip, cheap, low power, used in embedded devices.
What does an assembler do?	Converts assembly language into machine code that a specific processor can execute.
Four parts of an assembly statement?	Label (ends with `:`), mnemonic, operand(s), comment (begins with `;`).
Macro vs procedure?	Macro: no CALL/RET, code generated each call, more memory, <10 instructions. Procedure: uses CALL/RET, code generated once, less memory, >10 instructions.
Timer vs counter?	Counter increments on an **external** pulse (max rate 1/24 of oscillator); timer increments every **machine cycle** from the internal clock (max rate 1/12).
The 7 SDLC stages?	Planning & requirement analysis, defining requirements (SRS), design, development, testing, deployment, maintenance.
Which stage produces the SRS?	Stage 2 — defining requirements.
Which stage ensures user acceptance?	Testing, specifically **acceptance testing**.
Waterfall disadvantages?	Cannot accept requirement changes; high risk; unsuitable for large complex projects; working software comes late.
How long is one Agile iteration?	**One to three weeks.**
Name four Agile methods.	Scrum, XP (eXtreme Programming), FDD, DSDM, Crystal, Lean.
The defining feature of the spiral model?	**Risk analysis** in every loop.
White box vs black box in one line?	White box: internal code known, done by the **developer**, suits algorithm testing, slowest. Black box: only the GUI/functionality, done by the **test engineer**, fastest.
Verification vs validation?	Verification = "building the product right", static, no execution, QA. Validation = "building the right product", dynamic, code executes, QC.
Alpha vs beta testing?	Alpha: in-house, white + black box, before launch. Beta: real users, black box only, at marketing time.
Cohesion vs coupling — which is high, which is low?	Cohesion **high** (strength within a module); coupling **low** (dependence between modules).
Three types of feasibility?	Technical, operational (behavioural), economic.
Five steps of requirement engineering?	Feasibility study, elicitation & analysis, specification (SRS), validation, management.
DMAIC vs DMADV?	DMAIC (Define, Measure, Analyse, Improve, Control) improves an **existing** process; DMADV (. . . Design, Verify) creates a **new** design.
Singleton pattern?	A class with only one instance and a global point of access. Early or lazy instantiation. Saves memory; used in logging, caching, thread pools, configuration.

Last word

You are not behind because you are weak — you are behind because nobody showed you which 55 pages actually matter. Now somebody has. Do Part 3 until K-maps feel boring, and this question bank until you answer before you finish reading.